Leakage-Free Enhanced Private Set Union for Balanced and Unbalanced Scenarios

Qiang Liu
Chung-Ang University
Seoul, Republic of Korea
liuqiang0321@gmail.com

JaeYoung Bae
Chung-Ang University
Seoul, Republic of Korea
qo990102@gmail.com

JoonWoo Lee*
Chung-Ang University
Seoul, Republic of Korea
jwlee2815@cau.ac.kr

Abstract

Private Set Union (PSU) enables two parties to compute the union of their input sets without revealing anything else. Depending on set sizes, PSU is studied in balanced and unbalanced settings. Tu et al. (USENIX Security 2025) presented state-of-the-art enhanced PSU (ePSU) protocols under a unified framework in both settings, achieving enhanced security by preventing during-execution leakage. However, we observe that directly applying hash-to-bin on input sets within their framework introduces potential privacy risks. Moreover, the communication of their unbalanced ePSU still scales with the larger set size, rather than being linear in only the smaller set size. In this work, we address these open problems.

- We employ a combination of oblivious pseudorandom function (OPRF) and shuffling to mitigate the potential privacy leakage that arises when directly applying the hash-to-bin within the framework of Tu et al. (USENIX Security 2025). Building upon this, we further optimize their balanced ePSU protocol by leveraging a bidirectional oblivious key-value store (OKVS). Compared with the corrected version of Tu et al.'s balanced ePSU, ours achieves a $1.1-3.0\times$ shrinking in communication and a $1.2-1.6\times$ speedup in runtime.
- We design the first unbalanced ePSU whose communication is linear solely in the smaller set size. Since no hash-to-bin is used, it is inherently free from the associated privacy leakage. With the smaller set size fixed at 2^{10}, ours reduces communication by $1.5-45.8\times$ compared with corrected version of Tu et al.'s unbalanced ePSU, while achieving $1.3-6.7\times$ runtime speedups.

Keywords

enhanced private set union; balanced setting; unbalanced setting; during-execution leakage

1 Introduction

As a cryptographic protocol, private set union (PSU) enables two parties, the sender holding set X and the receiver holding set Y, to securely compute the union $X \cup Y$ without revealing anything else. PSU serves as an important privacy preserving tool for computing the union of sets, with significant applications in many areas such as security risk assessment [1–3], IP blacklist and vulnerability data aggregation [4–6], and private ID matching [7–9].

Depending on the size discrepancy between the two input sets, PSU can be categorized into balanced and unbalanced settings. In the balanced PSU, both parties hold sets of equal set size, i.e., $|X| = |Y| = n$, and the protocol aims for communication and computation that scale linearly with n. In contrast, the unbalanced PSU considers asymmetric set sizes, where one party holds a much smaller set of size $m \ll n$. In this case, the design goal is to achieve linear communication with respect to the smaller set size m, while remaining the independent of the larger set size n. Both settings are essential in practice. The balanced model captures the theoretical efficiency limits under symmetric interactions, while the unbalanced model aligns with real-world deployments characterized by resource-constrained clients and powerful servers. Hence, considering both models is crucial for designing PSU protocols that are both theoretically sound and practically efficient.

Balanced PSU. In 2019, Kolesnikov et al. [10] proposed the first scalable balanced PSU protocol. The core primitive is the reverse private membership test (RPMT), which privately decides for each item in X whether it belongs to Y. The indication bit is then used as the input to oblivious transfer (OT) [11] so that the receiver obtains a dummy value when $x_i \in Y$ and obtains x_i otherwise, and finally forms the union $X \cup Y$. Subsequent works [8, 12, 13] improved performance by optimizing RPMT.

However, in 2024, Jia et al. [14] showed that the RPMT+OT framework inherently suffers from during-execution leakage (DEL). Briefly, by invoking RPMT, it leaks the intersection-size of sets X and Y before protocol completion. In practice, this disclosure may lead to unintended leakage about the items in the intersection $X \cap Y$. For instance, a corrupted receiver could abort the execution after learning the intersection size (e.g., blaming a network failure) and request to rerun the protocol. By slightly modifying its input set across multiple executions and observing the resulting intersection sizes, the receiver can infer partial information about the intersection items even under standard rate-limiting mechanisms. This makes such leakage particularly insidious in practical deployments. To mitigate DEL, they designed a balanced enhanced PSU (ePSU) with enhanced security, but the complexity is superlinear. Recently, in 2025, Tu et al. [16] presented the first linear balanced ePSU by introducing permuted non-membership conditional randomness generation (pnMCRG).

In addition, Chandran et al. [17] observed that directly applying the hash-to-bin on input sets to compute the set union can lead to potential privacy leakage, which we call hash-based leakage. Briefly, once the receiver obtains $X \setminus Y$, it can infer the items in its simple hash table that do not match for an intersection. They proposed encrypting sets using OPRF, applying the encrypted sets within hash-to-bin, and employing shuffling to randomize the ordering, thereby mitigating this privacy leakage. However, their protocol also suffers from DEL. Evidently, several prior works [8, 10, 12, 16] are susceptible to hash-based leakage, whereas the scheme of [13] inherently avoids it by not employing the hash-to-bin in its design.

Unbalanced PSU. Tu et al. [18] designed the first unbalanced PSU protocol using fully homomorhic encryption (FHE) [19–21]

*Corresponding author.

that achieves sublinear communication complexity in the larger set size. Then, Zhang et al. [22] proposed an improved scheme based on sparse oblivious key-value store (sparse OKVS) [29], which enhances online-phase performance compared to [18] while maintaining sublinear communication complexity in the larger set size. However, the setup phase incurs substantial overhead that amortizes poorly. In addition, the protocols by Tu et al. [18] and Zhang et al. [22] both rely on the RPMT+OT framework and therefore suffer from DEL. Recently, in 2025, Tu et al. [16] introduced the first unbalanced ePSU. However, the communication still remains sublinear in the larger set size, falling to achieve linear that only depends on the smaller set size. Furthermore, protocols in [16, 18, 22] relying on directly applying hash-to-bin on input sets would cause hash-based leakage.

1.1 Motivation

Our work is driven by two main motivations. First, existing PSU protocols suffer from privacy leakage to some extent, stemming either from DEL or hash-based leakage. Second, in the unbalanced setting, the communications of existing protocols remain tied to the larger set size, failing to achieve linear that relies solely on the smaller set size. Therefore, we pose the following challenge:

Is it possible to design enhanced PSU protocol that, in the balanced setting, achieves both linear computation and communication, and in the unbalanced setting, achieves communication that is linear solely in the smaller set size, while being provably secure against privacy leakage from both during-execution leakage and hash-based leakage?

1.2 Contribution

In this work, we give an affirmative answer to the above challenges through the following results.

Revisiting state-of-the-art ePSU. We conduct a review of the balanced and unbalanced ePSU protocols in [16].

- **Security.** Both protocols are constructed under the same framework. In their framework, the sender and the receiver employ hash-to-bin, directly using their input sets to build cuckoo hash table and simple hash table, respectively. After obtaining $X \setminus Y$, the receiver can use it and simple hash table to deduce which items in Y are not in the intersection. This results in additional privacy leakage, which we eliminate by incorporating OPRF and shuffling.
- **Performance.** Their balanced ePSU performs computing and sending on many dummy entries, adding redundant cost. For their unbalanced ePSU, since each bin in the simple hash table has capacity $O(\log n)$, the receiver must perform expensive homomorphically evaluation on a per-bin polynomial and return ciphertexts, which yields $O(m \log n)$ communication; hence, the communication remains coupled to the larger set size.

Construction in the balanced ePSU. We address the hash-based leakage identified in the framework of [16] and, based on the revised framework, design a more efficient balanced ePSU protocol. Roughly, both parties invoke OPRF to encrypt their input set, ensuring that identical items yield the same ciphertext while distinct items yield different ones. Then, the encrypted sets are processed using hash-to-bin and bidirectional OKVS, the parties derive per equality conditional randomness that are equal if and only if the item in i-th cuckoo hash bin belongs to the i-th simple hash bin. Then, to achieve uniform distribution, we generate random values for empty cuckoo hash bins. In our design, cuckoo hash table is not padded, and OKVS is built only over occupied bins, so no redundancies are encoded or transmitted at this stage, which reduces the communication and computation. Fllowing, the same procedure as in [16], both parties sequentially invoke permuted equality conditional randomness generation (pECRG), non-equality conditional randomness generation (nECRG), and one-time pad, allowing the receiver to finally obtain the union set $X \cup Y$.

Construction in the unbalanced ePSU. We design a new primitive, oblivious equality-conditional randomness generation (OECRG), and combine it with nECRG to build the first unbalanced ePSU whose communication is strictly linear in the smaller set size only. Based on hash proof system (HPS) [31] and RSA assumption, OECRG obliviously generates per equality conditional randomness that are equal if and only if $x_i \in \mathsf{Y}$ and different otherwise. Then both parties sequentially invoke nECRG, and one-time pad, allowing the receiver to finally obtain the union set $X \cup Y$.

Evaluation. We implement our balanced and unbalanced ePSU protocols, and compare them against state-of-the-art, respectively. The results are summarized as follows:

- In the balanced setting, compared to the corrected balanced ePSU in [16], ours achieves a $1.1 - 3.0\times$ shrinking in communication and $1.2 - 1.6\times$ speedup in runtime, depending on the set sizes and network environment.
- In the unbalanced setting, fixing the smaller set size at 2^{10}, ours reduces communication cost by $1.5 - 45.8\times$ compared to the corrected unbalanced ePSU in [16], while achieving runtime speedups of $1.3 - 6.7\times$, depending on the set sizes and network environment.

1.3 Related Work

We review the previous scalable PSU protocols against semi-honest adversaries under both balanced and unbalanced settings. For clarity, let the sender hold set X and the receiver hold set Y. In the balanced setting $|\mathsf{X}| = |\mathsf{Y}| = n$, while in the unbalanced setting $|\mathsf{X}| = m \ll |\mathsf{Y}| = n$. The detailed analysis is given in Appendix D. Table 1 presents a comparison of our designed protocols with existing scalable PSU protocols.

2 Preliminaries

2.1 Notation

We let κ and λ denote the computational and statistical security parameters, respectively. The notation 1^κ is the string of κ ones. For any $n \in \mathbb{N}$, write $[n] = \{1, \ldots, n\}$. For a string $b \in \{0,1\}^\ell$, let b_i be its i-th bit. For a set $\mathsf{X} = \{x_1, \ldots, x_n\} \in (\{0,1\}^\ell)^n$, let $x_{i,j}$ denote the j-th bit of x_i. We use $\approx_C$ to denote computational indistinguishability and PPT for probabilistic polynomial-time. For strings x and y, $x \oplus y$ denotes the bitwise-XOR. The notation $r \xleftarrow{\$} \{0,1\}^\ell$ means r is sampled uniformly at random from $\{0,1\}^\ell$.

Protocols	Communication	Computation	DEL
[10]*	$O(n \log n)$	$O(n \log n \log\log n)$	Y
[8]*	$O(n \log n)$	$O(n \log n)$	Y
[12]*	$O(n \log n)$	$O(n \log n)$	Y
[13]	$O(n)$	$O(n)$	Y
[17]	$O(n)$	$O(n)$	Y
[14]*	$O(n \log n)$	$O(n \log n)$	N
Balanced in [16]*	$O(n)$	$O(n)$	N
Ours	$O(n)$	$O(n)$	N
[18]*	$O(m \log n)$	$O(n + m \log n)$	Y
[22]*	$O(m \log n)$	$O(n + m \log n)$	Y
Unbalanced in [16]*	$O(m \log n)$	$O(n + m \log n)$	N
Ours	$O(m)$	$O((m+n)\log n)$	N

Table 1: Comparisons of scalable PSU in the semi-honest model. "Y" indicates that during-execution leakage occurs; "N" indicates that no during-execution leakage occurs; "*" denotes the hash-based leakage caused by directly applying hash-to-bin on input sets.

2.2 Enhanced Private Set Union

A PSU protocol that incurs no during execution leakage is regarded as having an enhanced security, and such protocols are referred as enhanced PSU (ePSU). The functionality is shown in Figure 1.

Parameters:

- The functionality interacts with two parties, the sender $\mathcal{S}$ with set size n_1 and the receiver $\mathcal{R}$ with set size n_2, and the simulator *Sim*.

Functionalities:

(1) Initialize an ideal state $\text{state}_{\mathcal{U}} = \emptyset$ for party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$; if $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for input $\mathsf{X} = \{x_1, \ldots, x_{n_1}\}$ from $\mathcal{S}$, abort if $|\mathsf{X}| \neq n_1$; otherwise, update state $\text{state}_{\mathcal{S}} = \langle \mathsf{X} \rangle$, and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for input $\mathsf{Y} = \{y_1, \ldots, y_{n_2}\}$ from $\mathcal{R}$, abort if $|\mathsf{Y}| \neq n_2$; otherwise, update state $\text{state}_{\mathcal{R}} = \langle \mathsf{Y} \rangle$, and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon obtaining $\langle \text{Request}, \mathsf{OK} \rangle$ from *Sim*, compute $\mathsf{Z} = \mathsf{X} \cup \mathsf{Y}$, and add $\langle \text{Finished} \rangle$ to $\mathcal{S}$'s state $\text{state}_{\mathcal{S}}$ and $\langle \mathsf{Z} \rangle$ to $\mathcal{R}$'s state $\text{state}_{\mathcal{R}}$.
(5) Output Z to $\mathcal{R}$ and $\langle \text{Finished} \rangle$ to $\mathcal{S}$.

Figure 1: Enhanced private set union functionality $\mathcal{F}_{\text{ePSU}}^{n_1,n_2}$

2.3 Oblivious Pseudorandom Function

An oblivious pseudorandom function (OPRF) [27] allows the receiver to input $\{x_1, \ldots, x_n\}$ and obtains PRF values $\{F(key, x_1), \ldots, F(key, x_n)\}$, and the sender obtains a PRF key key. The definition of OPRF functionality is shown in Figure 2.

2.4 Oblivious Key-Value Store

A key-value store [28–30] is a data structure that maps a set of keys to their corresponding values, defined as follows:

Parameters:

- the sender $\mathcal{S}$ and the receiver $\mathcal{R}$.
- A PRF $F : \{0,1\}^{\ell} \times \{0,1\}^{\ell} \to \{0,1\}^{\kappa}$, where ℓ is the input length.

Functionalities:

(1) Wait for input $\{x_1, \ldots, x_n\}$ from $\mathcal{R}$.
(2) Sample a random PRF key key and compute $\{F(key, x_1), \ldots, F(key, x_n)\}$.
(3) Send $\langle \text{Request} \rangle$ to the simulator *Sim*.
(4) Upon obtaining $\langle \text{Request}, \mathsf{OK} \rangle$ from *Sim*, send the key key to $\mathcal{S}$ and $\{F(key, x_1), \ldots, F(key, x_n)\}$ to $\mathcal{R}$.

Figure 2: Oblivious pseudorandom function functionality $\mathcal{F}_{\text{OPRF}}$.

Definition 1. A key-value store (KVS) is parameterized by a set $\mathcal{K}$ of keys, a set $\mathcal{V}$ of values, and a set of functions H, and consists of two algorithms:

- $\mathsf{Encode}(\{(k_1, v_1), \ldots, (k_n, v_n)\})$: on input a set of (k_i, v_i) key-value pairs, and outputs an object $\mathsf{D} = (d_1, \ldots, d_m)$ (or, with statistically samll probability, an error indicator $\perp$).
- $\mathsf{Decode}(\mathsf{D}, k)$: on input an object $\mathsf{D} = (d_1, \ldots, d_m)$, a key k, and outputs a value v.

Correctness A KVS is correct if, for all $\mathsf{A} \subseteq \mathcal{K} \times \mathcal{V}$ with distinct keys, $(k, v) \in \mathsf{A}$ and $\perp \neq \mathsf{D} \leftarrow \mathsf{Encode}(\mathsf{A}) \implies \mathsf{Decode}(\mathsf{D}, k) = v$.
Obliviousness A KVS is an oblivious KVS (OKVS) if, for all distinct $\{k_1^0, \ldots, k_n^0\}$ and all distinct $\{k_1^1, \ldots, k_n^1\}$, if Encode does not output $\perp$ for $\{k_1^0, \ldots, k_n^0\}$ or $\{k_1^1, \ldots, k_n^1\}$, then $\{\mathsf{D} | v_i \leftarrow \mathcal{V}, i \in [n], \mathsf{Encode}_{\mathsf{H}}(\{(k_1^0, v_1), \ldots, (k_n^0, v_n)\})\} \approx_C \{\mathsf{D} | v_i \leftarrow \mathcal{V}, i \in [n], \mathsf{Encode}_{\mathsf{H}}(\{(k_1^1, v_1), \ldots, (k_n^1, v_n)\})\}$
Randomness. For any $A = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ and for any $x^* \notin \{x_1, \ldots, x_n\}$, the output of $\mathsf{Decode}(D, x^*)$ is statistically indistinguishable to that of uniform distribution over $\mathcal{V}$, where $\mathsf{D} \leftarrow \mathsf{Encode}(A)$.

2.5 Permuted Equality Conditional Randomness Generation

Permuted equality conditional randomness generation (pECRG) [16] enables the sender inputs a permutation $\pi : [n] \to [n]$ and $\{c_1, \ldots, c_n\}$, and the receiver inputs $\{c_1^*, \ldots, c_n^*\}$. Finally, the functionality returns the $\{u_1, \ldots, u_n\}$ and $\{u_1^*, \ldots, u_n^*\}$ to the sender and the receiver, respectively, where for $i \in [n]$, $u_i = u_i^*$ if $c_{\pi^{-1}(i)} = c^*_{\pi^{-1}(i)}$; otherwise $u_i \neq u_i^*$. The definition of pECRG functionality is shown in Figure 3.

2.6 Non-Equality Conditional Randomness Generation

Non-equality conditional randomness generation (nECRG) [16] enables the sender and the receiver input $\{t_1, \ldots, t_n\}$ and $\{\tilde{t}_1, \ldots, \tilde{t}_n\}$, respectively. Finally, the sender and the receiver obtain $\{r_1, \ldots, r_n\}$ and $\{\tilde{r}_1, \ldots, \tilde{r}_n\}$, respectively, where $r_i = \tilde{r}_i$ if $t_i \neq \tilde{t}_i$; otherwise, $r_i \neq \tilde{r}_i$. The definition of nECRG functionality is shown in Figure 4.

Parameters:

- Two parties: the sender $\mathcal{S}$ and the receiver $\mathcal{R}$.

Functionalities:

(1) Initialize an ideal state $\text{state}_{\mathcal{U}} = \emptyset$ for party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$; if $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for input $\{c_1, \ldots, c_n\}$ and a permutation $\pi : [n] \to [n]$ from $\mathcal{S}$ and update state $\text{state}_{\mathcal{S}} = \langle \{c_i\}_{i\in[n]}, \pi \rangle$, and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for input $\{c_1^*, \ldots, c_n^*\}$ from $\mathcal{R}$ and update state $\text{state}_{\mathcal{R}} = \langle \{c_i^*\}_{i\in[n]} \rangle$, and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon obtaining $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, generate two random sets $\{u_1, \ldots, u_n\}$ and $\{u_1^*, \ldots, u_n^*\}$, where $u_i = u_i^*$ if $c_{\pi^{-1}(i)} = c^*_{\pi^{-1}(i)}$; otherwise, $u_i \neq u_i^*$.
(5) Add $\langle \{u_i\}_{i\in[n]} \rangle$ to state $\text{state}_{\mathcal{S}}$ and $\langle \{u_i^*\}_{i\in[n]} \rangle$ to state $\text{state}_{\mathcal{R}}$.
(6) Output $\langle \{u_i\}_{i\in[n]} \rangle$ to $\mathcal{S}$ and $\langle \{u_i^*\}_{i\in[n]} \rangle$ to $\mathcal{R}$.

Figure 3: Permuted equality conditional randomness generation functionality $\mathcal{F}_{\text{pECRG}}$

Parameters:

- the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator *Sim*.

Functionalities:

(1) Initialize an ideal state $\text{state}_{\mathcal{U}} = \emptyset$ for party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$; if $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for input $\{t_1, \ldots, t_n\}$ from $\mathcal{S}$ and update state $\text{state}_{\mathcal{S}} = \langle \{t_i\}_{i\in[n]} \rangle$, and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for input $\{\tilde{t}_1, \ldots, \tilde{t}_n\}$ from $\mathcal{R}$ and update state $\text{state}_{\mathcal{R}} = \langle \{\tilde{t}_i\}_{i\in[n]} \rangle$, and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon obtaining $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, generate two random sets $\{r_1, \ldots, r_n\}$ and $\{\tilde{r}_1, \ldots, \tilde{r}_n\}$, where $r_i = \tilde{r}_i$ if $t_i \neq \tilde{t}_i$; otherwise, $r_i \neq \tilde{r}_i$.
(5) Add $\langle \{r_i\}_{i\in[n]} \rangle$ to state $\text{state}_{\mathcal{S}}$ and $\langle \{\tilde{r}_i\}_{i\in[n]} \rangle$ to state $\text{state}_{\mathcal{R}}$.
(6) Output $\langle \{r_i\}_{i\in[n]} \rangle$ to $\mathcal{S}$ and $\langle \{\tilde{r}_i\}_{i\in[n]} \rangle$ to $\mathcal{R}$.

Figure 4: Non-equality conditional randomness generation functionality $\mathcal{F}_{\text{nECRG}}$.

2.7 Hash Proof System

Hash proof system (HPS) [31–34] is a PPT algorithm consisting of an array HPS = $\{\mathbf{D}, \mathbf{L}, \mathbf{W}, R, \mathbf{HSK}, \mathbf{HPK}, \pi, \alpha, H, F\}$. $\mathbf{D}$ denotes a non-empty field and $\mathbf{L}$ is a proper subset of $\mathbf{D}$. $\mathbf{W}$ is a witness space, and a statement $\mathbf{x} \in \mathbf{L}$ if there exists a witness $\mathbf{w} \in \mathbf{W}$ for $(\mathbf{x}, \mathbf{w}) \in R$, where R is an efficiently computable binary relation. $\mathbf{HSK}$ and $\mathbf{HPK}$ denote private key space and public key space. π is the proof space, and $\mathbf{HPS}$ allows one to generate a valid proof π for a statement $\mathbf{x}$. A valid proof π proves $\mathbf{x} \in \mathbf{L}$ by using $\mathbf{w}$ and $\mathbf{hpk}$ or only using $\mathbf{hsk}$. Each statement $\mathbf{x}$ is sampled indistinguishably from inside and outside $\mathbf{L}$. α is chosen efficiently computable map such that $\alpha : \mathbf{HSK} \to \mathbf{HPK}$. HPS efficiently samples a private key $\mathbf{hsk}$ randomly from $\mathbf{HSK}$ and chooses a family of efficiently computable function $H_\kappa : \{\pi | \pi = H_\kappa(\mathbf{hsk}, \mathbf{x}), \mathbf{hsk} \in \mathbf{HSK}, \mathbf{x} \in \mathbf{L}\}$. HPS chooses an efficiently computable map $F_\kappa : \{\pi | \pi = F_\kappa(\mathbf{x}, \mathbf{w}, \mathbf{hpk}), \mathbf{hpk} \in \mathbf{HPK}, \mathbf{x} \in \mathbf{L}, \mathbf{w} \in \mathbf{W}\}$. HPS consists of three algorithms.

- Key generation algorithm $I(1^\kappa)$: On input security parameter κ, a key generation algorithm outputs a pair of public and private keys $(\mathbf{hpk}, \mathbf{hsk})$ of length κ, and $\alpha : \alpha(\mathbf{hsk}) = \mathbf{hpk}$.
- Private evaluation algorithm $H_\kappa(\cdot)$: On input security parameter κ, for any $(\mathbf{x}, \mathbf{w}) \in R$, a private evaluation algorithm efficiently outputs $\pi = H_\kappa(\mathbf{hsk}, \mathbf{x})$ using a PPT algorithm.
- Public evaluation algorithm $F_\kappa(\cdot)$: On input security parameter κ, for any pair $(\mathbf{x}, \mathbf{w}) \in R$, a public evaluation algorithm efficiently outputs $\pi = F_\kappa(\mathbf{hpk}, \mathbf{x}, \mathbf{w})$ using a PPT algorithm.

Then, $\forall \mathbf{x} \in \mathbf{L}, H_\kappa(\mathbf{hsk}, \mathbf{x}) = F_\kappa(\mathbf{hpk}, \mathbf{x}, \mathbf{w})$. Moreover, HPS satisfies two properties, universality and smoothness. Universality means that the entropy of π should be large enough for fixed $\mathbf{x} \in \mathbf{D}/\mathbf{L}$ and $\mathbf{hpk}$. Smoothness means that the distribution of π must be close to the uniform distribution of the proof space π for $\mathbf{x} \in \mathbf{D}/\mathbf{L}$ and $\mathbf{hsk}$. Definitions are as follows.

Definition 2. HPS satisfies universality if the probability of an adversary $\mathcal{A}$ successfully generating the proof π from guessing $\mathbf{x}$ using a PPT algorithm without private key $\mathbf{hsk}^*$ or witness $\mathbf{w}^*$ is negligible.

$$\Pr[H_\kappa^{\mathcal{A}}(\mathbf{hsk}^*, \mathbf{x}) = \pi \wedge F_\kappa^{\mathcal{A}}(\mathbf{hpk}, \mathbf{x}, \mathbf{w}^*) = \pi] < \epsilon$$

where ϵ is a negligible parameter, and $\mathbf{hsk}^*$ and $\mathbf{w}^*$ are chosen randomly in the private key and witness space.

Definition 3. HPS satisfies smoothness if the probability of an adversary $\mathcal{A}$ successfully distinguishing the proof π and π^* from the statements $\mathbf{x}$ and $\mathbf{x}^*$, $\mathbf{x} \neq \mathbf{x}^*$, using a PPT algorithm with private key $\mathbf{hsk}^*$ or witness $\mathbf{w}^*$ is negligible.

$$\Pr[H_\kappa^{\mathcal{A}}(\mathbf{hsk}, \mathbf{x}) = \pi \wedge F_\kappa^{\mathcal{A}}(\mathbf{hsk}^*, \mathbf{x}^*) = \pi^*] < \epsilon(\kappa),$$

$$\Pr[F_\kappa^{\mathcal{A}}(\mathbf{hpk}, \mathbf{x}, \mathbf{w}) = \pi \wedge F_\kappa^{\mathcal{A}}(\mathbf{hpk}, \mathbf{x}^*, \mathbf{w}^*) = \pi^*] < \epsilon(\kappa),$$

where $\alpha(\mathbf{hsk}) = \mathbf{hpk}$ and $\epsilon(\kappa)$ is a negligible function.

2.8 Hash-to-bin

Cuckoo hashing scheme [36] uses γ hash functions $h_1, \ldots, h_\gamma : \{0,1\}^* \to [\beta]$ to map n items to bins in a hash table of size $\beta = \epsilon n$, and each bin contains only one item. When an item x is inserted, $x||k$ (where k denotes the corresponding hash label) is initially hashed into one of the bins $B_{h_1(x)}, \ldots, B_{h_\gamma(x)}$. If all designated bins are occupied, then randomly selects one of the occupied bins, evicts the item currently in that bin, and re-inserts it into another bin determined by a different hash function. This process is repeated iteratively until either a vacant bin is found, or the number of evictions reaches a predefined threshold, indicating that insertion into the regular hash table is infeasible. In such cases, the item is placed into a small auxiliary space called the "stash," reserved for overflow items. Mathematically, by appropriately tuning the parameters γ and ϵ, the stash size can be minimized to zero in most

practical scenarios while maintaining a failure probability as low as $2^{-\lambda}$ [35]. **Simple hashing** scheme [36] enables γ hash functions $h_1, \cdots, h_\gamma : \{0,1\}^* \to [\beta]$ to map n items into β bins $B_1, \cdots, B_\beta$. Each item $x_i||j$ ($i \in [n]$) is added to bins $B_{h_j(x_i)}$, where $j \in [\gamma]$.

3 Balanced Enhanced Private Set Union

In this section, we first analyze the hash-based leakage in the framework of [16] arising from directly applying hash-to-bin on input sets and propose a correction to address this issue. Subsequently, based on the revised framework, we propose an improved balanced ePSU protocol by incorporating a birdirectional OKVS.

3.1 Analyzing and Mitigating Leakage in the Framework of [16]

Before analyzing the potential hash-based leakage in the framework of [16], we first briefly review the details of its protocol, as summarized below.

1. The sender uses $\gamma = 3$ hash functions to construct cuckoo hash table $X_C = \{X_C[i]\}_{i\in[\beta]}$ and fills empty bins with the dummy value $\perp$, where each bin $X_C[i]$ contains only one item and there is a index $k_{x_i} \in [\gamma]$ such that $X_C[h_{k_{x_i}}(x_i)] = x_i||k_{x_i}$ for each x_i;
2. The receiver uses the same $\gamma = 3$ hash functions to construct simple hash table. For each $k_i \in [\gamma]$, the value $y_i||k_i$ is placed into the bin $Y_{h_{k_i}(y_i)}$. Finally, there are β bins $Y_1, \ldots, Y_\beta$, and each bin Y_i contains $|Y_i|$ items.
3. The sender and receiver invoke pnMCRG, where the sender inputs X_C and a permutation $\pi : [\beta] \to [\beta]$ and the receiver inputs $Y_1, \ldots, Y_\beta$. After execution, the sender and receiver obtain $\{u_1, \ldots, u_\beta\}$ and $\{v_1, \ldots, v_\beta\}$, respectively, where if $X_C[\pi(i)] \notin Y_{\pi(i)}$, then $u_i = v_i$; otherwise $u_i \neq v_i$.
4. The sender computes $\{c_i\}_{i\in[\beta]}$ and sends it to receiver, where $c_i = u_i \oplus (X_C[\pi^{-1}[i]||\mathsf{H}(X_C[\pi^{-1}[i])$, and H is pre-agreed hash function.
5. The receiver sets $Z = \{\mathsf{Y}\}$. For each $i \in [\beta]$, the receiver computes $m_i||m_i' = v_i \oplus c_i$ and sets $Z = Z \cup m_i$ if $m_i' = \mathsf{H}(m_i)$ and $m_i' \neq \perp$. Finally, the receiver obtains the union set Z.

According to step 5 of their protocol, the receiver computes $m_i||m_i'$ and checks whether $m_i = h(m_i')$ and $m_i \neq \perp$. All m_i satisfying these conditions are collected to form the set $\mathsf{X} \setminus \mathsf{Y}$, and the union is then obtained as $Z = (\mathsf{X} \setminus \mathsf{Y}) \cup \mathsf{Y}$. However, the receiver can infer the additional information from $\mathsf{X} \setminus \mathsf{Y}$, thereby causing hash-based leakage. Specifically, a corrupted receiver can exploit this leakage as follows: for each $y \in \mathsf{Y}$, the receiver can locally compute its candidate bin positions $\mathsf{Pos}(y) = \{h_1(y), \ldots, h_\gamma(y)\}$. Upon obtaining $\mathsf{X}\setminus\mathsf{Y}$, it can use hash functions to inspect the bins in which the items of $\mathsf{X}\setminus\mathsf{Y}$ reside. If all of y's candidate bins are already occupied by distinct items from $\mathsf{X}\setminus\mathsf{Y}$, the receiver concludes that $y \notin \mathsf{X}$. Hence, even though $\mathsf{X}\setminus\mathsf{Y}$ is necessary for correctness, its hash structure reveals negative membership information about the sender's set. This attack requires no access to X_C itself, only to the public hash functions and the observable distribution of $\mathsf{X}\setminus\mathsf{Y}$ across bins.

Formally, let B_i denote the set of items mapped to bin i by the sender, where

$$B_i = \{x \in \mathsf{X} : \exists j \in [\gamma], h_j(x) = i\}.$$

After learning $\mathsf{X}\setminus\mathsf{Y}$, the receiver knows the occupied bins

$$\mathrm{Occ}_X = \{i : \exists x \in \mathsf{X}\setminus\mathsf{Y}, h_j(x) = i\}.$$

Then for any $y \in \mathsf{Y}$,

$$\forall j,\ h_j(y) \in \mathrm{Occ}_X \Longrightarrow y \notin \mathsf{X}.$$

Thus the receiver leans a leakage function

$$\mathcal{L}_{\mathcal{R}}^{\mathrm{ePSU}}(\mathsf{X},\mathsf{Y}) = \{y \in \mathsf{Y} : \forall j,\ h_j(y) \in \mathrm{Occ}_X\},$$

which equals the subset of Y that can be certified as not in X. In addition, [17] shows that if the receiver knows $|\mathsf{X}\cap\mathsf{Y}|$, $|\mathsf{X}\setminus\mathsf{Y}|$ and $|\mathsf{Y}|$, the probability that it can compute some $y \notin \mathsf{X}\cap\mathsf{Y}$ is $\mathrm{pr} = 1 - \frac{|\mathsf{X}\cap\mathsf{Y}|}{|\mathsf{Y}|} \cdot (\frac{2\beta}{3(\beta-1)})^{|\mathsf{X}\setminus\mathsf{Y}|}$. Then, we can know $\Pr[\mathcal{L}_{\mathcal{R}}^{\mathrm{ePSU}}(\mathsf{X},\mathsf{Y})] = \mathrm{pr}$. Evidently, $|\mathsf{X}\cap\mathsf{Y}|$ equals the number of items satisfying $m_i \neq h(m_i')$ minus $(\beta - |\mathsf{X}|)$, where $(\beta - |\mathsf{X}|)$ denotes the number of dummy values $\perp$ in the cuckoo hash table, and $|\mathsf{X} \setminus Y|$ corresponds to the number of items satisfying $m_i = h(m_i')$ and $m_i \neq \perp$. Consequently, after completing step 5, the receiver can directly derive these quantities.

To address this issue, Chandran et al. [17] proposed encrypting the input sets using an OPRF before applying the hash-to-bin technique, thereby replacing the direct hash-to-bin operation on plaintext inputs. This approach mitigates the hash-based leakage that arises when the receiver exploits $\mathsf{X} \setminus \mathsf{Y}$ to infer items $y \notin \mathsf{X}$. In addition, they incorporate a shuffling mechanism to randomize the protocol output, preventing the receiver from inferring whether the sender's i-th item corresponds to the i-th bin in its simple hash table. We adopt the suggestion of [17] and revise the framework of [16] accordingly, as described below.

(1) the sender and the receiver invoke OPRF, where the receiver inputs $\mathsf{Y} = \{y_1, \ldots, y_n\}$ and obtains $\{F(key, x_1), \ldots, F(key, x_n)\}$, while the sender obtains a PRF key key.
(2) The sender uses $\gamma = 3$ hash functions to construct cuckoo hash tables ($X_C = \{X_C[i]\}_{i\in[\beta]}, \tilde{X}_C = \{\tilde{X}_C[i]\}_{i\in[\beta]}$) and fills empty bins with the dummy values ($\perp, \perp'$), where each cuckoo hash bin contains only one item and there is a index $k_{x_i} \in [\gamma]$ such that $X_C[h_{k_{x_i}}(F(key, x_i))] = x_i||k_{x_i}$ and $\tilde{X}_C[h_{k_{x_i}}(F(key, x_i))] = F(key, x_i)||k_{x_i}$ for each x_i;
(3) The receiver uses the same $\gamma = 3$ hash functions to construct simple hash table. For each $k_i \in [\gamma]$, the value $F(key, y_i)||k_i$ is placed into the bin $Y_{h_{k_i}(F(key,y_i))}$. Finally, there are β bins $Y_1, \ldots, Y_\beta$, and each bin Y_i contains $|Y_i|$ items.
(4) The sender and receiver invoke pnMCRG, where the sender inputs $\tilde{X}_C$ and a permutation $\pi : [\beta] \to [\beta]$ and the receiver inputs $Y_1, \ldots, Y_\beta$. After execution, the sender and receiver obtain $\{u_1, \ldots, u_\beta\}$ and $\{v_1, \ldots, v_\beta\}$, respectively, where if $X_C[\pi(i)] \notin Y_{\pi(i)}$, then $u_i = v_i$; otherwise $u_i \neq v_i$.
(5) The sender computes $\{c_i\}_{i\in[\beta]}$ and sends it to receiver, where $c_i = u_i \oplus (X_C[\pi^{-1}[i]||\mathsf{H}(X_C[\pi^{-1}[i])$, and H is pre-agreed hash function.
(6) The receiver sets $Z = \{\mathsf{Y}\}$. For each $i \in [\beta]$, the receiver computes $m_i||m_i' = v_i \oplus c_i$ and sets $Z = Z \cup m_i$ if $m_i' = \mathsf{H}(m_i)$ and $m_i' \neq \perp$. Finally, the receiver obtains the union set Z.

Note that in [16], the functionality pnMCRG is defined as follows: the sender inputs a permutation $\pi : [n] \to [n]$ and a set $\{x_1, \ldots, x_n\}$, while the receiver inputs n sets $Y_1, \ldots, Y_n$. After execution, the sender and receiver obtain $u_1, \ldots, u_n$ and $v_1, \ldots, v_n$,

respectively, such that for each $i \in [n]$, if $x_{\pi^{-1}(i)} \notin Y_{\pi^{-1}(i)}$, then $u_i = v_i$, and otherwise $u_i \neq v_i$. Therefore, pnMCRG inherently achieves the shuffling functionality via the input permutation π, and thus no additional shuffle operation is required in our revision. In our revised framework, for each $y \in \mathsf{Y}$, the receiver's candidate bin positions $\mathsf{Pos}(y) = \{h_1(F(key, y)), \ldots, h_\gamma(F(key, y))\}$. Each bin B_i is defined as

$$B_i = \{x \in \mathsf{X} : \exists j \in [\gamma], h_j(F(key, x)) = i\},$$

and the corresponding occupancy set is defined as

$$\mathsf{Occ}_X = \{i : \exists x \in \mathsf{X} \backslash \mathsf{Y}, h_j(F(key, x)) = i\}.$$

Since the OPRF ensures that the receiver has no knowledge of the PRF key key, it cannot compute $F(key, x)$ for any $x \in \mathsf{X} \setminus \mathsf{Y}$. Consequently, the receiver is unable to construct or evaluate Occ_X, i.e., it cannot determine whether $h_j(F(key, y)) \in \mathsf{Occ}_x$. As a result, the receiver cannot infer which items belong to Y but not to X.

Additionally, in the revised ePSU framework, besides the two communication phases in the original protocol, namely, the invocation of pnMCRG and the sender's transmission of ciphertexts $\{c_i\}_{i\in[\beta]}$, an additional interaction is introduced through the OPRF call. The security of the OPRF protocol ensures that this interaction is computationally indistinguishable for any corrupted party, while the other two phases have already been proven computationally indistinguishable in [16]. Hence, the revised protocol is secure in the semi-honest model.

3.2 Our Improved balanced ePSU based on Bidirectional OKVS

After addressing the potential hash-based privacy leakage in the framework of [16], we further analyze its balanced ePSU protocol and propose an optimization based on a bidirectional OKVS construction.

In their framework, hash-to-bin and pnMCRG serve as the core components. The pnMCRG can be further decomposed into three building blocks: membership conditional randomness generation (MCRG), pECRG, and nECRG. The functionality of MCRG is defined as follows: the sender inputs $x_1, \ldots, x_n$, while the receiver inputs n sets $Y_1, \ldots, Y_n$. After execution, the sender and receiver obtain $m_1, \ldots, m_n$ and $\alpha_1, \ldots, \alpha_n$, respectively, such that for each $i \in [n]$, if $x_i \in Y_i$, then $m_i = \alpha_i$; otherwise, $m_i \neq \alpha_i$. Our optimization mainly focuses on improving the hash-to-bin and MCRG components.

As described by our revised framework in section 3.1, the sender and receiver construct the cuckoo hash tables $(X_C, \tilde{X}_C)$ and simple hash table $(Y_1, \ldots, Y_\beta)$ in steps (1–3), respectively. In step (4), they invoke pnMCRG, where the sender provides π and $\tilde{X}_C$, while the receiver provides $Y_1, \ldots, Y_\beta$. Internally, pnMCRG first executes MCRG and then sequentially runs pECRG and nECRG before producing its outputs. Since our improvements primarily focus on the MCRG component of their pnMCRG framework, we begin with a brief description of their MCRG protocol. Figure 5 illustrates its workflow within the balanced ePSU protocol.

As shown in Figure 5, the sender and receiver first invoke a batch OPRF (bOPRF) [37]. The sender inputs $\{\tilde{X}_C[i]\}_{i\in[\beta]}$, after which the receiver obtains β PRF keys $\{k_i\}_{i\in[\beta]}$ and the sender obtains $\{w_i\}_{i\in[\beta]}$, where $w_i = F(k_i, \tilde{X}_C[i])$. Next, the receiver constructs an OKVS data structure $D = \mathsf{Encode}(\{P_i\}_{i\in[\beta]})$, where

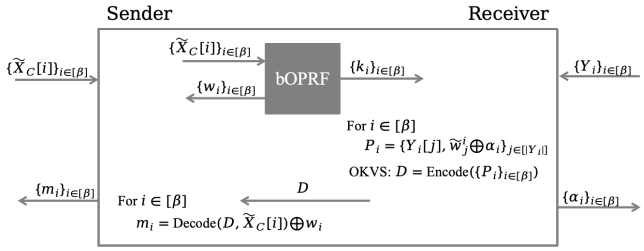

Figure 5: The protocol workflow of MCRG of balanced ePSU in [16]. Note: $w_i = F(k_i, \tilde{X}_C[i])$; $\tilde{w}_j^i = F(k_i, Y_i[j])$.

each $P_i = \{(Y_i[j], \tilde{w}_j^i \oplus \alpha_i)\}_{j\in|Y_i|}$ and α_i is a random value, $\tilde{w}_j^i = F(k_i, Y_i[j])$. The receiver then sends D to the sender. For each $i \in [\beta]$, the sender computes $m_i = \mathsf{Decode}(D, \tilde{X}_C[i]) \oplus w_i$. Finally, the sender and receiver obtain their respective MCRG outputs: $\{m_i\}_{i\in[\beta]}$ and $\{\alpha_i\}_{i\in[\beta]}$. By construction, if $\tilde{X}_C[i] \in Y_i$, then $m_i = \mathsf{Decode}(D, \tilde{X}_C[i]) \oplus w_i = w_i \oplus \alpha_i \oplus w_i = \alpha_i$; otherwise, $\mathsf{Decode}(D, \tilde{X}_C[i])$ produces a random value, making m_i a random, meaningless output.

Since the hash-to-bin is employed, the sender's cuckoo hash tables inevitably contain $\beta - n$ padded dummy slots. In the MCRG protocol illustrated in Figure 5, the bOPRF is invoked over all β slots, causing these dummy positions to incur additional computation and communication overhead. According to the bOPRF protocol described in [37], which represents the most widely used implementation, processing β inputs of κ bits each results in approximately $2\kappa\beta$ bits of communication (excluding the base OT cost), of which $2\kappa(\beta-n)$ bits are redundant. Our optimization aims to exclude these dummy slots from the active execution path while still ensuring the correctness of the outputs $\{m_i\}_{i\in[\beta]}$ and $\{\alpha_i\}_{i\in[\beta]}$. The dummy slots are instead treated as local placeholders, thereby avoiding unnecessary cost; in the final output, the corresponding m_i values can simply be replaced with random values.

Building upon this optimization insight, we observe that the bOPRF in Figure 5 is in fact unnecessary. If both parties attach identical random tags to the same keys, then locally XORing the returned values cancels out the matching tags, yielding a fixed, known value; for non-matching items, the results remain pseudorandom. This observation motivates our bidirectional OKVS-based approach: the sender writes an individual key–value pair $(\tilde{X}_C[i], r_i)$ for each real item $\tilde{X}_C[i] \neq \perp'$ to construct an OKVS object D. The receiver then decodes its candidate keys $F(k_i, y_i)$ via $\mathsf{Decode}(\mathrm{D}, F(k_i, y_i))$, XORs the outputs with local randomness, re-encodes the results into another OKVS object D^*, and sends it back. Finally, the sender decodes using its own $\tilde{X}_C[i]$ (where $\tilde{X}_C[i] \neq \perp'$) and XORs the corresponding r_i to determine the outcome. This design eliminates the need for bOPRF entirely, relying only on the obliviousness and randomness of OKVS. The overall workflow of our optimized scheme is illustrated in Figure 6.

As shown in Figure 6, the sender constructs an OKVS data structure $\mathrm{D} = \mathsf{Encode}(M)$, where $M = \{(\tilde{X}_C[i], r_i) \mid \widetilde{X}_C[i] \neq \perp'\}$ is the set of key–value pairs corresponding to all non-dummy entries. The sender then sends D to the receiver. For each $i \in [n]$ and $j \in [\gamma]$, the receiver computes $s_i^j = \mathsf{Decode}(\mathrm{D}, w_i^* \| j)$, where

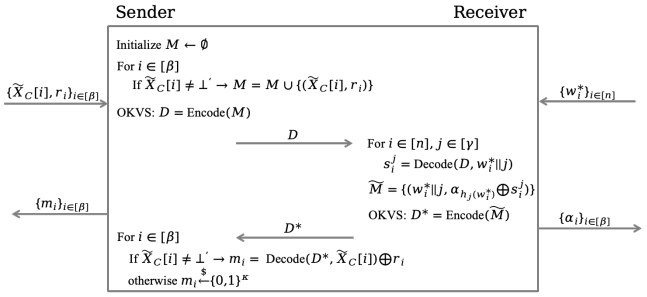

Figure 6: Our improved workflow based on MCRG

$w_i^* = F(\text{key}, y_i)$. If $w_i^*\|j$ equals a key contained in M, then s_i^j equals the corresponding value in M; otherwise, s_i^j is a random value. The receiver then constructs a new set of key–value pairs $\tilde{M} = \{(w_i^*\|j, \alpha_{h_j(w_i^*)} \oplus s_i^j)\}_{i\in[n],j\in[\gamma]}$, generates an OKVS object $\mathsf{D}^* = \text{Encode}(\widetilde{M})$, and sends it to the sender. For each $i \in [\beta]$, if $\tilde{X}_C[i] \neq \perp'$, the sender computes $m_i = \text{Decode}(\mathsf{D}^*, \widetilde{X}_C[i]) \oplus r_i$; otherwise, $m_i \xleftarrow{\$} \{0,1\}^\kappa$. If $\tilde{X}_C[i]$ equals a key in $\widetilde{M}$, for instance $\tilde{X}_C[i] = w_k^*\|k'$, then $s_k^{k'} = r_i$, which implies $\text{Decode}(\mathsf{D}^*, \tilde{X}_C[i]) = \alpha_{h_{k'}(w_k^*)} \oplus s_k^{k'} = \alpha_i \oplus r_i \implies m_i = \alpha_i$. Otherwise, if $\tilde{X}_C[i]$ does not match any key in $\widetilde{M}$, then $\text{Decode}(\mathsf{D}^*, \tilde{X}_C[i])$ yields a random value, and consequently m_i is also random. Therefore, our improved scheme correctly outputs $\{m_i\}_{i\in[\beta]}$ and $\{\alpha_i\}_{i\in[\beta]}$. Moreover, unlike the original construction, our scheme does not require the receiver to explicitly build a simple hash table. Instead, during the generation of $\tilde{M}$, the receiver simply uses the hash functions h_j ($j \in [\gamma]$) to compute the index $h_j(w_i^*)$ for each random value $\alpha_{h_j(w_i^*)}$, thereby determining the corresponding position efficiently.

From a performance perspective, the original protocol shown in Figure 5 incurs $2\beta\kappa$ bits of communication from the bOPRF and $(1+\sigma)n\gamma\kappa$ bits from the receiver's transmission of the OKVS data structure D. Hence, the total communication cost is $2\beta\kappa + (1+\sigma)n\gamma\kappa$ bits. Note that for the encoding of key-value pairs, the OKVS in [30] achieves an optimal value of $\sigma = 0.03$. In contrast, our improved scheme requires only two OKVS transmissions, resulting in a total communication of $(\gamma+1)(1+\sigma)n\kappa$ bits. This yields a communication reduction of $2\beta\kappa - (1+\sigma)n\kappa$ bits. For example, setting $\sigma = 0.05$, $\gamma = 3$, and $\beta = 1.27n$, our optimization achieves a reduction of approximately $1.51n\kappa$ bits.

After the sender and the receiver obtain $\{m_i\}_{i\in[\beta]}$ and $\{\alpha_i\}_{i\in[\beta]}$, we simply follow the subsequent steps of our revised ePSU framework to complete the remaining procedure. Specifically, the sender and receiver first run pECRG with inputs a permutation π, $\{m_1, \ldots, m_\beta\}$ and $\{\alpha_1, \ldots, \alpha_\beta\}$. They obtain outputs $\{c_1, \ldots, c_\beta\}$ and $\{c_1^*, \ldots, c_\beta^*\}$, respectively, where $c_i = c_i^*$ if $m_{\pi^{-1}(i)} = \alpha_{\pi^{-1}(i)}$, and $c_i \neq c_i^*$ otherwise. They then run nECRG on inputs $\{c_1, \ldots, c_\beta\}$ and $\{c_1^*, \ldots, c_\beta^*\}$ to obtain $\{u_1, \ldots, u_\beta\}$ and $\{v_1^*, \ldots, v_\beta^*\}$, respectively, where $u_i \neq v_i$ if $c_i = c_i^*$, and $u_i = v_i$ otherwise. Sender then sends $\{c_i\}_{i\in[\beta]}$ to receiver, where $c_i = u_i \oplus (\mathsf{X}_C[\pi^{-1}(i)]||\mathsf{H}(\mathsf{X}_C[\pi^{-1}(i)]))$. Receiver computes $m_i||m_i^* = c_i \oplus v_i$, and sets $\mathsf{Z} = \mathsf{Z} \cup \{m_i\}$ if $m_i^* = \mathsf{H}(m_i)$; otherwise, nothing. Finally, the receiver outputs the union $\mathsf{Z} \cup \mathsf{Y}$.

3.3 Protocol Details of Our Balanced ePSU

We give our improved formal balanced ePSU protocol that realizes the functionality $\mathcal{F}_{\text{ePSU}}^{n,n}$ in Figure 7.

Security. We state the security of our $\Pi_{\text{ePSU}}^{\text{Bal}}$ in Theorem 3.1, and the security proof is given in Appendix A.

Theorem 3.1. *The protocol Π_{ePSU}^{Bal} shown in Figure 7 securely computes $\mathcal{F}_{ePSU}^{n,n}$ functionality shown in Figure 1 in the $\{\mathcal{F}_{nECRG}, \mathcal{F}_{pCERG}\}$-hybrid model, against semi-honest adversaries.*

Complexity Analysis. We give the theoretical complexities of our $\Pi_{\text{ePSU}}^{\text{Bal}}$ in Table 2.

	Π_{OPRF}	Bir. OKVS	Π_{nECRG}	Π_{pCERG}	Ciphertexts
Comp.	$O(n)$	$O(\gamma \cdot n)$	$O(\epsilon \cdot n)$	$O(\epsilon \cdot n)$	$O(\epsilon \cdot n)$
Comm.	$O(n)$	$O(\gamma \cdot n)$	$O(\epsilon \cdot n)$	$O(\epsilon \cdot n)$	$O(\epsilon \cdot n)$

Table 2: The theoretical complexities of $\Pi_{\text{ePSU}}^{\text{Bal}}$. "Bir. OKVS" means "Birectional OKVS".

4 Unbalanced Enhanced Private Set Union

In this section, we briefly review the unbalanced ePSU protocol of [16], identify its performance bottlenecks, and present our remedy. In short, we construct our unbalanced ePSU by composing the newly introduced primitive oblivious equality conditional randomness generation (OECRG) with nECRG. The OECRG functionality enables the oblivious generation of equality conditional randomness based on the membership relation between two inputs. We construct an OECRG protocol suitable for such unbalanced scenarios under the HPS and RSA assumptions, achieving linear communication complexity in the smaller set size. Finally, we provide the full details of our unbalanced ePSU protocol.

4.1 Oblivious Equality Conditional Randomness

In the unbalanced setting of [16], the protocol still follows the ePSU framework described in section 3.1, except that the MCRG block inside pnMCRG differs from its counterpart in the balanced case. We observe that the overall communication complexity of their unbalanced ePSU protocol is $O(m \log n)$ rather than the ideal $O(m)$, which mainly stems from the combined design of the hash-to-bin and their MCRG protocol. Before presenting our improvements, we first briefly review the MCRG protocol in their unbalanced setting. Figure 8 illustrates the detailed structure of this protocol.

As illustrated in Figure 8, the sender inputs $\{\tilde{X}_C[i]\}_{i\in[\beta]}$ to invoke the bOPRF with the receiver. After the execution, the sender obtains $\{w_i\}_{i\in[\beta]}$ and the receiver obtains $\{k_i\}_{i\in[\beta]}$. For each $i \in [\beta]$, the sender encrypts w_i using the FHE scheme to obtain the ciphertext $\text{ct}_i = \text{Enc}(w_i)$, and sends the ciphertexts $\{\text{ct}_i\}_{i\in[\beta]}$ to the receiver. The receiver constructs, for each $i \in [\beta]$, a polynomial function $F_i(x) = f_i(x) + \alpha_i$ such that $f_i(F(k_i, Y_i[\cdot])) = 0$. The receiver then performs homomorphic evaluation over ct_i using the coefficients of $F_i(x)$, obtaining the ciphertext c_i. The receiver sends the ciphertexts $\{\mathsf{c}_i\}_{i\in[\beta]}$ to the sender, who decrypts them to obtain $\{m_i\}_{i\in[\beta]}$, where $m_i = \text{Dec}(\mathsf{c}_i)$. Consequently, the sender and the receiver respectively obtain the MCRG outputs $\{m_i\}_{i\in[\beta]}$ and $\{\alpha_i\}_{i\in[\beta]}$, which are then used in the subsequent computation.

Parameters:

- Two parties: the sender $\mathcal{S}$, the receiver $\mathcal{R}$.
- An OKVS scheme (Encode, Decode).
- Hash functions $h_1, \ldots, h_\gamma : \{0,1\}^\ell \to [\beta]$.
- A cuckoo hash table based on $h_1, \ldots, h_\gamma$, with $\beta = \epsilon \cdot n$ bins, stash size is 0.
- A collision-resistant hash function: $\mathsf{H} : \{0,1\}^\ell \to \{0,1\}^{\ell_1}$, where $\ell_1 \geq \lambda + 2\log \epsilon n$.

Input of the sender $\mathcal{S}$: $\mathsf{X} = \{x_1, \ldots, x_n\} \in (\{0,1\}^\ell)^n$.
Input of the receiver $\mathcal{R}$: $\mathsf{Y} = \{y_1, \ldots, y_n\} \in (\{0,1\}^\ell)^n$.

Protocol:

(1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{OPRF}}$:
- $\mathcal{R}$ inputs $\mathsf{Y} = \{y_1, \ldots, y_n\}$.
- $\mathcal{S}$ and $\mathcal{R}$ obtain key key and $\{w_1^*, \ldots, w_n^*\}$, respectively, where $w_i^* = F(key, y_i)$.

(2) The sender uses $\gamma = 3$ hash functions to construct cuckoo hash tables ($X_C = \{X_C[i]\}_{i\in[\beta]}, \tilde{X}_C = \{\tilde{X}_C[i]\}_{i\in[\beta]}$) and fills empty bins with the dummy values $(\perp, \perp')$, where each cuckoo hash bin contains only one item and there is a index $k_{x_i} \in [\gamma]$ such that $X_C[h_{k_{x_i}}(F(key, x_i))] = x_i || k_{x_i}$ and $\tilde{X}_C[h_{k_{x_i}}(F(key, x_i))] = F(key, x_i) || k_{x_i}$ for each x_i;

(3) $\mathcal{S}$ chooses β random values $\{r_1, \ldots, r_\beta\} \xleftarrow{\$} \{0,1\}^\kappa$. $\mathcal{S}$ sets $M = \{(\tilde{X}_C[i], r_i) \mid \tilde{X}_C[i] \neq \perp'\}$ and computes an OKVS structure $\mathsf{D} = \mathsf{Encode}(M)$. Then, $\mathcal{S}$ sends D to $\mathcal{R}$.

(4) $\mathcal{R}$ randomly chooses β values $\{a_1, \ldots, a_\beta\} \xleftarrow{\$} \{0,1\}^\kappa$ and computes the set $\{s_1^j, \ldots, s_n^j\}_{j\in[\gamma]}$, where $s_i^j = \mathsf{Decode}(\mathsf{D}, w_i^* || j)$ for $i \in [n]$. $\mathcal{R}$ sets $\tilde{M} = \{(w_i^* \| j,\ \alpha_{h_j(w_i^*)} \oplus s_i^j)\}_{i\in[n], j\in[\gamma]}$ and computes an OKVS structure $\mathsf{D}^* = \mathsf{Encode}(\tilde{M})$. Then, $\mathcal{R}$ sends D^* to $\mathcal{S}$.

(5) $\mathcal{S}$ computes the set $\{m_1, \ldots, m_\beta\}$, where for $i \in [\beta]$, $m_i = \mathsf{Decode}(\mathsf{D}^*, \mathsf{X_C}[i]) \oplus r_i$ if $\mathsf{X_C}[i] \neq 0$; otherwise $m_i \xleftarrow{\$} \{0,1\}^\kappa$.

(6) $\mathcal{S}$ chooses a permutation function $\pi : [\beta] \to [\beta]$. $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{pECRG}}$
- $\mathcal{S}$ inputs $\{m_1, \ldots, m_\beta\}$ and π, and $\mathcal{R}$ inputs $\{a_1, \ldots, a_\beta\}$.
- $\mathcal{S}$ and $\mathcal{R}$ obtain $\{c_1, \ldots, c_\beta\}$ and $\{c_1^*, \ldots, c_\beta^*\}$, respectively, where $c_i = c_i^*$ if $m_{\pi^{-1}(i)} = a_{\pi^{-1}(i)}$; otherwise, $c_i \neq c_i^*$.

(7) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{pCERG}}$:
- $\mathcal{S}$ inputs $\{c_1, \ldots, c_\beta\}$, and $\mathcal{R}$ inputs $\{c_1^*, \ldots, c_\beta^*\}$.
- $\mathcal{S}$ and $\mathcal{R}$ obtain $\{u_1, \ldots, v_\beta\}$, $\{v_1, \ldots, v_\beta\}$, respectively, where $u_i = v_i^*$ if $c_i \neq c_i^*$; otherwise, $u_i \neq v_i$.

(8) $\mathcal{S}$ computes and sends $\{c_i\}_{i\in[\beta]} = \{u_i \oplus (X_C[\pi^{-1}(i)] || \mathsf{H}(X_C[\pi^{-1}(i)]))\}_{i\in[\beta]}$ to $\mathcal{R}$.

(9) $\mathcal{R}$ sets $\mathsf{Z} \leftarrow \{\mathsf{Y}\}$. For $i \in [\beta]$, $\mathcal{R}$ computes $m_i || m^* = c_i \oplus v_i$, $\mathcal{R}$ sets $\mathsf{Z} = \mathsf{Z} \cup \{m_i\}$ if $m_i^* = \mathsf{H}(m_i)$.

(10) $\mathcal{R}$ outputs $\mathsf{Z} = \mathsf{Y} \cup \mathsf{Z}$ and $\mathcal{S}$ outputs $\langle \mathsf{Finished} \rangle$.

Figure 7: Our balanced enhanced private set union protocol $\Pi_{\text{ePSU}}^{\text{Bal}}$

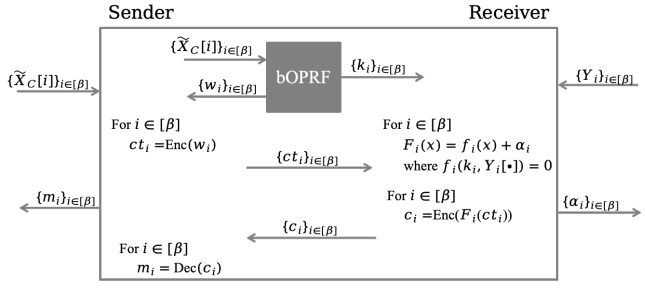

Figure 8: The protocol workflow of MCRG of unbalanced ePSU in [16]. Note: $w_i = F(k_i, \tilde{X}_C[i])$; $\tilde{w}_j^i = F(k_i, Y_i[j])$.

In their unbalanced ePSU protocol, the receiver distributes its items into simple hash bins $\{Y_i\}_{i\in[\beta]}$, where the load of each bin grows with n, i.e., $O(\log n)$. Since the cost of homomorphic evaluation $\mathsf{c}_i = \mathsf{Enc}(F_i(\mathsf{ct}_i))$ increases with the bin size, the communication cost of sending the homomorphically evaluated ciphertexts $\{\mathsf{c}_i\}_{i\in[\beta]}$ expands proportionally to $O(m \log n)$. As a result, the overall communication complexity of the unbalanced ePSU protocol remains coupled with the larger set size n, failing to achieve the desired linear $O(m)$ dependence on the smaller set size. Moreover, although their ePSU framework suffers from potential hash-based leakage, this issue can be mitigated by introducing an OPRF. However, in the unbalanced setting, the receiver with input size n must interact with the sender during the OPRF execution, which itself incurs an $O(n)$ communication overhead. Therefore, the total communication complexity of their unbalanced ePSU increases from $O(m \log n)$ to $O(n + m \log n)$.

It is evident that achieving $O(m)$ communication complexity is not feasible under the OPRF + hash-to-bin. To this end, we introduce a new scheme that eliminates the reliance on both OPRF and hash-to-bin. In brief, we introduce OECRG, which generates equality conditional randomness by obliviously checking whether each $x_i \in \mathsf{X}$ also exists in Y. Unlike previous approaches that first use hash-to-bin to locate potential matches of x_i in Y and then invoke equality-check-based randomness generation, our method directly performs oblivious equality checking. This not only avoids the redundant generation of $(\beta - n)$ equality conditional randomness caused by hash-to-bin, but, due to the oblivious nature of OECRG,

Parameters:

- Two parties: the sender $\mathcal{S}$ and the receiver $\mathcal{R}$.

Functionalities:

(1) Initialize an ideal state $\text{state}_{\mathcal{U}} = \emptyset$ for party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$; if $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for input $\mathsf{X} = \{x_1, \ldots, x_m\}$ from $\mathcal{S}$ and update state $\text{state}_{\mathcal{S}} = \langle \mathsf{X} \rangle$, and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for input $\mathsf{Y} = \{y_1, \ldots, y_n\}$ from $\mathcal{R}$ and update state $\text{state}_{\mathcal{R}} = \langle \mathsf{Y} \rangle$, and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon obtaining $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, generate two random sets $\{R_1, \ldots, R_m\}$ and $\{R_1^*, \ldots, R_m^*\}$, where $R_i = R_i^*$ if $x_i \in \mathsf{Y}$; $R_i \neq R_i^*$, otherwise.
(5) Add $\langle \{R_i\}_{i\in[m]} \rangle$ to state $\text{state}_{\mathcal{S}}$ and $\langle \{R_i^*\}_{i\in[m]} \rangle$ to state $\text{state}_{\mathcal{R}}$.
(6) Output $\langle \{R_i\}_{i\in[m]} \rangle$ to $\mathcal{S}$ and $\langle \{R_i^*\}_{i\in[m]} \rangle$ to $\mathcal{R}$.

Figure 9: Oblivious equality conditional randomness generation functionality $\mathcal{F}_{\text{OECRG}}$

also eliminates the need for the subsequent random permutation step (i.e., the pECRG invocation).

We give the functionality of OECRG in Figure 9. In this functionality, the sender inputs the set $\mathsf{X} = \{x_1, \ldots, x_m\}$ and the receiver inputs the set $\mathsf{Y} = \{y_1, \ldots, y_n\}$. Finally, the functionality returns the output $\{R_1, \ldots, R_m\}$ and $\{R_1^*, \ldots, R_m^*\}$ to the sender and the receiver, respectively, where for $i \in [m]$, $R_i = R_i^*$ if and only if $x_i \in Y$; otherwise $R_i \neq R_i^*$.

Our OECRG design is inspired by the unbalanced PSI protocol of Zhao et al. [34], which achieves linear $O(m)$ communication. Their construction relies on the HPS and RSA assumptions to ensure both correctness and security. In brief, the sender holds $\mathsf{X} = \{x_1, \ldots, x_m\}$ and the receiver holds $\mathsf{Y} = \{y_1, \ldots, y_n\}$ for each $x_i \in \mathsf{X}$ and they interact so that the receiver computes $g^{\prod_{j\in[n]} z_{i,j}} \bmod N_i$, where

$$z_{i,j} = p_i + \Delta(x_i - y_j) \bmod N,$$

p_i is a prime factor of $\Phi(N_i)$, $N > N_i$, and Δ is random value. The receiver then sends this value to the sender, who checks whether

$$\left(g^{\prod_{j\in[n]} z_{i,j}}\right)^{\Phi(N_i)/p_i} \bmod N_i = 1$$

to determine whether x_i belongs to the intersection. Clearly, if $x_i = y_k$, then $z_{i,k} = p_i$, and thus

$$g^{\prod_{j\in[n]} z_{i,j}} \bmod N_i = g^{z_{i,1}\cdots z_{i,k-1} p_i z_{i,k+1}\cdots z_{i,n}} \bmod N_i,$$

which implies

$$\left(g^{\prod_{j\in[n]} z_{i,j}}\right)^{\Phi(N_i)/p_i} \bmod N_i = (g^0)^{z_{i,1}\cdots z_{i,k-1} z_{i,k+1}\cdots z_{i,n}} \bmod N_i = 1.$$

Otherwise, if $x_i \neq y_j$ for all $j \in [n]$, the resulting value is uniformly random and independent of x_i.

Building upon the above idea, we observe that if the receiver chooses a random value r_i and computes $g^{\prod_{j\in[n]} z_{i,j}} \cdot g^{r_i} \bmod N_i$ and sends it to the sender, then the sender can compute

$$\left(g^{\prod_{j\in[n]} z_{i,j}} \cdot g^{r_i}\right)^{\Phi(N_i)/p_i} \bmod N_i$$

and return $g^{\Phi(N_i)/p_i} \bmod N_i$ to the receiver. The receiver then computes

$$\left(g^{\Phi(N_i)/p_i}\right)^{r_i} \bmod N_i.$$

Clearly, if $x_i \in \mathsf{Y}$, we have

$$\left(g^{\prod_{j\in[n]} z_{i,j}} \cdot g^{r_i}\right)^{\Phi(N_i)/p_i} \bmod N_i = g^{r_i \Phi(N_i)/p_i} \bmod N_i,$$

which is equal to the receiver's value $\left(g^{\Phi(N_i)/p_i}\right)^{r_i} \bmod N_i$. Otherwise, it is random and independent of r_i.

Hence, the sender's value $\left(g^{\Phi(N_i)/p_i}\right)^{r_i} \bmod N_i$ and the receiver's value $\left(g^{\prod_{j\in[n]} z_{i,j}} \cdot g^{r_i}\right)^{\Phi(N_i)/p_i} \bmod N_i$ form a pair of equality conditional randomness.

Next, we present the complete protocol flow of our OECRG construction, as described below. The parameters used in the protocol are summarized in Table 3.

Parameters	Meaning
$\mathbf{pk} \in (\mathbb{Z}_N^*)^{1\times\ell}$	Public key
$\mathbf{sk} \in (\mathbb{Z}_{N-1})^{1\times s}$	Private key
$\mathbf{A} \in (\mathbb{Z}_N^*)^{s\times\ell}$	Random matrix
$\mathbf{B} \in (\mathbb{Z}_{N-1}^*)^{1\times\ell}$	Random vector
N	Large prime
ℓ	Bit length of items of input sets.
$\{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\}$	Random vectors
$\mathbf{D}/\mathbf{L}$	Vector set, a subset of $\mathbf{D}$, $\mathbf{D} = (\mathbb{Z}_{\mathbb{N}}^*)^{1\times s}$
$\mathbf{L}$	A subset of $\mathbf{D}$, satisfying $\mathbf{A}\cdot\mathbf{B} \in \mathbf{L}$
p_i, q_i	Large primes
N_i	Large primes, $N_i = (2p_i+1)(2q_i+1) < N$
g_i	Generator of $\mathbb{Z}_{N_i}^*$

Table 3: Introduction to parameter meanings.

First, the receiver samples a secret key $\mathbf{sk} \xleftarrow{\$} \mathbf{HSK}$, and computes the corresponding public key $\mathbf{pk} = \mathbf{sk}\cdot\mathbf{A} \bmod N$. Then, it randomly generates $\{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\} \xleftarrow{\$} \mathbf{D}/\mathbf{L}$, and publishes the public parameters $(\mathbf{pk}, \mathbf{A}, \{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\}, N)$; The sender holds parameters $\{p_i, q_i, N_i, g_i\}_{i\in[m]}$, where p_i is a prime factor of $\varphi(N_i)$ for each $i \in [m]$. The sender publishes the public parameters $\{N_i, g_i\}_{i\in[m]}$.

Following, For each $i \in [m]$, the sender randomly selects $\mathbf{w}_i \in (\mathbb{Z}_{N-1}^*)^{1\times\ell}$ such that $\mathbf{pk}\cdot\mathbf{w}_i = p_i$, and computes the ciphertext $\mathbf{c}_i^S = \sum_{k=1}^{\ell} x_{i,k}\mathbf{a}_k + \mathbf{A}\mathbf{w}_i \bmod N$. The sender transmits $\{\mathbf{c}_1^S, \ldots, \mathbf{c}_m^S\}$ to the receiver. After that, for $i \in [m]$, the receiver computes and sends $M_i = (g_i)^{\prod_{j=1}^n z_{i,j}} \cdot g_i^{r_i} \bmod N_i$ to the sender, where

$$\begin{aligned} z_{i,j} &= \mathbf{sk}\cdot\mathbf{c}_i^S - \mathbf{sk}\textstyle\sum_{k=1}^{\ell} y_{j,k}\mathbf{a}_k \bmod N \\ &= p_i + \mathbf{sk}\textstyle\sum_{k=1}^{\ell}(x_{i,k}\mathbf{a}_k - y_{j,k}\mathbf{a}_k) \bmod N, \end{aligned}$$

and r_i is a random value. For $i \in [m]$, the sender computes

$$M_i^* = g_i^{t_i} \bmod N_i \text{ and } R_i = \mathsf{H}(M_i^{t_i}) \bmod N_i$$

where $t_i = \Phi(N_i)/p_i$ and H is a collusion-resistant hash function. Finally, the sender returns $\{M_1^*, \ldots, M_m^*\}$ to the receiver, who computes

$$R_i^* = \mathsf{H}(M_i^{*r_i} \bmod N_i) = \mathsf{H}(g_i^{r_i t_i} \bmod N_i)$$

Notably, if $x_i \in \mathsf{Y}$, then $M_i^{t_i} = g_i^{r_i t_i} \bmod N_i \Longrightarrow R_i = R_i^*$; otherwise, M_i is random $\Longrightarrow R_i \neq R_i^*$.

Parameters:

- Two parties: the sender $\mathcal{S}$ and the receiver $\mathcal{R}$, set size m and n, where $m \ll n$.
- A collision-resistant hash function $\mathsf{H} : \{0,1\}^{\ell} \to \{0,1\}^{\ell_1}$, where $\ell_1 \geq \lambda + 2\log m$

Input of the sender $\mathcal{S}$: $\mathsf{X} = \{x_1, \ldots, x_m\} \in (\{0,1\}^{\ell})^m$.
Input of the receiver $\mathcal{R}$: $\mathsf{Y} = \{y_1, \ldots, y_n\} \in (\{0,1\}^{\ell})^n$.
Protocol:

(1) $\mathcal{R}$ holds $\{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\} \xleftarrow{\$} \mathbf{D}/\mathbf{L}$ and $\{\mathbf{sk}, \mathbf{pk}, \mathbf{A}, N\}$ generated by $\mathsf{HPS.KeyGen}(1^\kappa)$, where $\mathbf{pk} = \mathbf{sk} \cdot \mathbf{A} \bmod N$. $\mathcal{R}$ publishes $(\mathbf{pk}, \mathbf{A}, \{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\}, N)$.
(2) $\mathcal{S}$ holds $\{p_i, q_i, N_i, g_i\}_{i\in[m]}$ generated by $\mathsf{RSA.KeyGen}(1^\kappa)$, where $p_i | \Phi(N_i)$ and g_i is a generator of $\mathbb{Z}^*_{N_i}$. $\mathcal{S}$ publishes $\{N_1, \ldots, N_m\}$ and $\{g_1, \ldots, g_m\}$.
(3) For $i \in [m]$, $\mathcal{S}$ operates as follows:
   - Select a ℓ-dimensional witness vector $\mathbf{w}_i \leftarrow (\mathbb{Z}^*_{N-1})^{1\times\ell}$, s.t. $\mathbf{pk} \cdot \mathbf{w}_i = p_i$.
   - Compute $\mathbf{c}_i^{\mathcal{S}} = \sum_k x_{i,k}\mathbf{a}_k + \mathbf{A}\mathbf{w}_i \mod N$, where $k \in [\ell]$.

   Then, $\mathcal{S}$ sends $\{\mathbf{c}_1^{\mathcal{S}}, \ldots, \mathbf{c}_m^{\mathcal{S}}\}$ to $\mathcal{R}$.
(4) For $j \in [n]$, $\mathcal{R}$ computes $\mathbf{c}_j^{\mathcal{R}} = \mathbf{sk} \sum_k y_{j,k}\mathbf{a}_k \mod N$, where $k \in [\ell]$.
(5) $\mathcal{R}$ randomly selects $(r_1, \ldots, r_m)$. For $i \in [m]$, $\mathcal{R}$ operates as follows:
   - Compute $\tilde{\mathbf{c}}_i^{\mathcal{S}} = \mathbf{sk} \cdot \mathbf{c}_i^{\mathcal{S}} \mod N$. For $j \in [n]$, compute $z_{i,j} = \tilde{\mathbf{c}}_i^{\mathcal{S}} - \mathbf{c}_j^{\mathcal{R}} \mod N$.
   - Compute $\mathsf{M}_i = g_i^{\prod_{j\in[n]} z_{i,j}} \cdot g_i^{r_i} \bmod N_i$

   Then, $\mathcal{R}$ sends $\{\mathsf{M}_1, \ldots, \mathsf{M}_m\}$ to $\mathcal{S}$.
(6) For $i \in [m]$, $\mathcal{S}$ sets $t_i = \Phi(N_i)/p_i$ and computes $M_i^* = g_i^{t_i} \mod N_i$ and $R_i = \mathsf{H}(M_i^{t_i} \mod N_i)$. $\mathcal{S}$ sends $\{M_1^*, \ldots, M_m^*\}$ to $\mathcal{R}$.
(7) For $i \in [m]$, $\mathcal{R}$ computes $R_i^* = \mathsf{H}(M_i^{*r_i} \mod N_i)$.
(8) $\mathcal{S}$ and $\mathcal{R}$ output $\{R_1, \ldots, R_m\}$ and $\{R_1^*, \ldots, R_m^*\}$, respectively.

Figure 10: Our oblivious equality conditional randomness generation protocol $\Pi_{\mathbf{OECRG}}$

Through the above processes, the sender obtains $\{R_1, \ldots, R_m\}$ and the receiver obtains $\{R_1^*, \ldots, R_m^*\}$ as their equality conditional randomness. Our formal OECRG protocol is shown in Figure 10.

Security. We state the security of our Π_{OECRG} in Theorem 4.1, and the security proof is given in Appendix B.

Theorem 4.1. *Assume the security of the HPS and RSA assumption. The protocol Π_{OECRG} shown in Figure 10 securely computes $\mathcal{F}_{OECRG}$ functionality shown in Figure 9 against semi-honest adversaries.*

4.2 Protocol Details of Our Unbalanced ePSU

We give the detailed description of our unbalanced ePSU as follows. First, the sender and the receiver invoke OECRG with input X and Y, then the sender and the receiver obtain the output $\{R_1, \ldots, R_m\}$ and $\{R_1^*, \ldots, R_i^*\}$, respectively. In this process, if $x_i \in Y$, then $R_i = R_i^*$; otherwise, $R_i \neq R_i^*$. Following, the sender and the receiver invoke nECRG with input $\{R_1, \ldots, R_m\}$ and $\{R_1^*, \ldots, R_i^*\}$, then the sender and the receiver obtain the output $\{t_1, \ldots, t_m\}$ and $\{t_1^*, \ldots, t_m^*\}$, respectively. In this process, if $R_i = R_i^*$, then $t_i \neq t_i^*$; otherwise, $t_i = t_i^*$. the sender sends $\{v_i\}_{i\in[\beta]} = \{t_i \oplus (x_i || \mathsf{H}(x_i))\}_{i\in[m]}$ to the receiver. the receiver computes $m_i || m_i^* = v_i \oplus t_i^*$ and sets $\mathsf{Z} = \mathsf{Z} \cup \{m_i\}$ if $m_i^* = \mathsf{H}(m_i)$; otherwise, nothing. Finally, the receiver outputs the union $\mathsf{Z} \cup \mathsf{Y}$. Clearly, our unbalanced ePSU does not involve any hash-to-bin, and thus is inherently free from the privacy leakage introduced by the hash-to-bin. The formal description of our unbalanced ePSU is shown in Figure 11.

Security. We state the security of our $\Pi_{\mathrm{ePSU}}^{\mathrm{Unb}}$ in Theorem 4.2, and the security proof is given in Appendix C.

Theorem 4.2. *The protocol Π_{ePSU}^{Unb} shown in Figure 11 securely computes $\mathcal{F}_{ePSU}^{m,n}$ functionality shown in Figure 1 against semi-honest adversaries in the ($\mathcal{F}_{OECRG}$, $\mathcal{F}_{nECRG}$)-hybrid model.*

Complexity Analysis. We give the theoretical complexities of our $\Pi_{\mathrm{ePSU}}^{\mathrm{Unb}}$ in Table 4.

	Π_{OECRG}	Π_{nECRG}	Ciphertexts
Comp.	$O((m+n)\log m)$	$O(m)$	$O(m)$
Comm.	$O(m)$	$O(m)$	$O(m)$

Table 4: The theoretical complexities of $\Pi_{\mathbf{ePSU}}^{\mathbf{Unb}}$. Note. by the FFT algorithm, the computational complexity of Π_{OECRG} can be reduced from $O(mn)$ to $O((m+n)\log n)$.

5 Experimental Setup

We experimentally evaluate the performance of our balanced and unbalanced ePSU protocols.

Benchmarking Environment. We implement our protocols in C++ on a MacBook Pro running a macOS 14.0, equipped with an Apple M3 Pro chip and 36GB RAM. We evaluate the protocols in two network settings: LAN (10Gbps bandwidth, 0.2ms RTT) and WAN (100Mbps, 10Mbps bandwidth, 80ms RTT).

Implementation Details. We set the computational security parameter $\kappa = 128$, the statistical security parameter $\lambda = 40$, and use $\gamma = 3$ hash functions to insert set X into the cuckoo hash table with bin size $b = \epsilon \cdot n = 1.27n$ in our balanced setting. We set the length of each item in the sets to 64-bit. We implement OPRF

Parameters:

- Two parties: the sender $\mathcal{S}$ and the receiver $\mathcal{R}$.
- A collision-resistant hash function $\mathsf{H} : \{0,1\}^{\ell} \to \{0,1\}^{\ell_1}$, where $\ell_1 \geq \lambda + 2\log m$.

Input of the sender $\mathcal{S}$: $\mathsf{X} = \{x_1, \ldots, x_m\} \in (\{0,1\}^{\ell})^m$.
Input of the receiver $\mathcal{R}$: $\mathsf{Y} = \{y_1, \ldots, y_n\} \in (\{0,1\}^{\ell})^n$.
Protocol:

(1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{OECRG}}$:
- $\mathcal{S}$ inputs X and $\mathcal{R}$ inputs Y.
- $\mathcal{S}$ and $\mathcal{R}$ obtain $\{R_1, \ldots, R_m\}$ and $\{R_1^*, \ldots, R_m^*\}$, respectively, where for $i \in [m]$, $R_i = R_i^*$ if $x_i \in \mathsf{Y}$; otherwise, $R_i \neq R_i^*$.

(2) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{nECRG}}$:
- $\mathcal{S}$ inputs $\{R_1, \ldots, R_m\}$ and $\mathcal{S}$ inputs $\{R_1^*, \ldots, R_m^*\}$.
- $\mathcal{S}$ and $\mathcal{R}$ obtain $\{t_1, \ldots, t_m\}$ and $\{t_1^*, \ldots, t_m^*\}$, respectively, where for $i \in [m]$, $t_i = t_i^*$ if $R_i \neq R_i^*$; otherwise, $t_i \neq t_i^*$.

(3) $\mathcal{S}$ computes and sends $\{v_i\}_{i\in[m]} = \{t_i \oplus (x_i || \mathsf{H}(x_i))\}_{i\in[m]}$ to $\mathcal{R}$.

(4) For $i \in [m]$, the receiver computer $m_i || m_i^* = v_i \oplus t_i^*$ and set $\mathsf{Z} = \mathsf{Z} \cup \{m_i\}$ if $m_i^* = \mathsf{H}(m_i)$.

(5) $\mathcal{R}$ outputs $\mathsf{Z} = \mathsf{Y} \cup \mathsf{Z}$ and $\mathcal{S}$ outputs $\langle \text{Finished} \rangle$.

Figure 11: Our unbalanced private set union protocol $\Pi_{\text{ePSU}}^{\text{Unb}}$

following the construction in [27] (code at [39]), OKVS based on [30] (code at [40]) and nECRG/pECRG based on [16] (code at [41]). The OECRG is built using the Cryptopp and NTL libraries (see [42], [43]), while hash functions, PRNG, and channel handling leverage OpenSSL, cryptoTools, and coproto libraries (see [44], [45], [46]).

5.1 Performance Evaluation in Balanced Setting

We implement our balanced ePSU and compare it against state-of-the-art balanced PSU with DEL [13] and balanced ePSU (PKE-based) [16]. We also implement the corrected version of balanced ePSU from [16]. The results of the communication and runtime comparison in the single thread are presented in Table 5 and Figure 12. The comparison with balanced ePSU in [16] is shown as follows.

- **Communication.** As shown in Figure 12 and Table 5, our balanced ePSU outperforms the balanced ePSU from [16] when the set size is up to 2^{15}. Beyond this point, the balanced ePSU of [16] gradually gains an advantage. However, our balanced ePSU avoids the hash-based leakage inherent in [16]. Furthermore, it is evident that our protocol consistently outperforms the corrected balanced ePSU of [16], as illustrated in Figure 12. For set sizes ranging from 2^{10} to 2^{20}, Table 5 shows that our protocol achieves approximately $1.1 - 3.0\times$ shrinking in communication. Although both our balanced ePSU and the corrected version of [16] exhibit linear communication complexity, our protocol benefits from a smaller constant factor, thereby maintaining its advantage even at larger scales.
- **Runtime.** As shown in Figure 12, our protocol outperforms both the balanced ePSU of [16] and its corrected version across different bandwidths and set sizes. Specially, compared with the corrected balanced ePSU, Table 5 shows that our protocol achieves $1.3 - 1.5\times$ speedup for set sizes ranging from 2^{10} to 2^{20}.

Size($\lvert X\rvert = \lvert Y\rvert$)	Protocols	Comm.(MB)	Total running time (s) with single thread		
			10 Gbps	100 Mbps	10 Mbps
2^{10}	[13] (DEL)	**0.188**	2.879	**4.625**	**4.568**
	[16]* (DEL-free)	1.310	0.252	6.602	7.192
	Corrected [16] (DEL-free)	1.436	0.315	8.077	9.263
	Our $\Pi_{\text{ePSU}}^{\text{Bal}}$ (DEL-free)	0.485	**0.249**	5.950	6.842
2^{12}	[13] (DEL)	**0.700**	3.912	**5.467**	**6.133**
	[16]* (DEL-free)	2.028	0.646	8.462	10.818
	Corrected [16] (DEL-free)	2.374	0.785	10.289	13.391
	Our $\Pi_{\text{ePSU}}^{\text{Bal}}$ (DEL-free)	1.330	**0.553**	7.223	9.974
2^{14}	[13] (DEL)	**2.726**	8.629	**11.121**	**12.879**
	[16]* (DEL-free)	5.202	3.299	15.839	21.035
	Corrected [16] (DEL-free)	6.434	3.787	18.230	25.917
	Our $\Pi_{\text{ePSU}}^{\text{Bal}}$ (DEL-free)	4.960	**3.045**	14.908	16.316
2^{16}	[13] (DEL)	**11.143**	27.555	32.489	34.287
	[16]* (DEL-free)	17.955	16.617	37.507	49.941
	Corrected [16]($DEL - free$)	22.767	18.524	41.812	59.864
	Our $\Pi_{\text{ePSU}}^{\text{Bal}}$ (DEL-free)	19.369	**12.671**	**28.601**	**43.078**
2^{18}	[13] (DEL)	**44.712**	100.076	112.925	**111.047**
	[16]* (DEL-free)	69.030	71.419	118.107	159.086
	Corrected [16] (DEL-free)	88.303	79.345	131.211	193.379
	Our $\Pi_{\text{ePSU}}^{\text{Bal}}$ (DEL-free)	77.905	**59.007**	**97.554**	152.616
2^{20}	[13] (DEL)	**179.061**	378.837	423.428	**447.859**
	[16]* (DEL-free)	277.402	300.834	418.054	660.375
	Corrected [16] (DEL-free)	355.082	337.076	468.481	792.119
	Our $\Pi_{\text{ePSU}}^{\text{Bal}}$ (DEL-free)	314.243	**251.045**	**353.388**	610.877

Table 5: Comparisons of communication and runtime between our ePSU and (PSU with DEL [13], ePSU [16], corrected ePSU in [16]) in balanced setting in single thread. 10Gbps bandwidth, 0.2ms RTT; 100Mbps and 10Mbps bandwidth, 80ms RTT; The best and the second results are marked in pink and blue, respectively. "*" denotes the privacy leakage caused by directly applying hash-to-bin on sets.

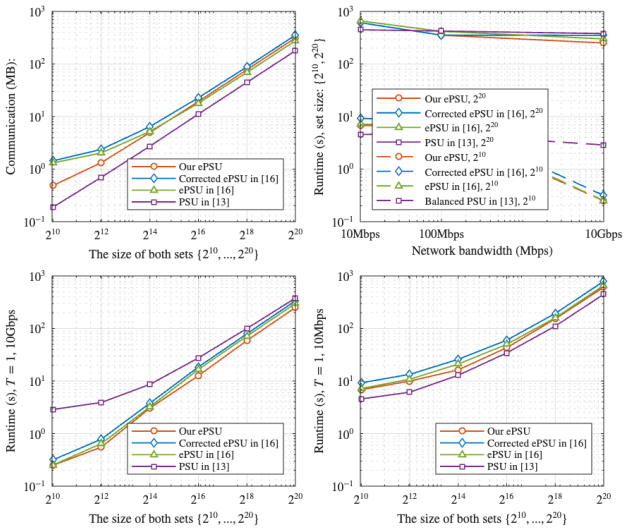

Figure 12: Comparisons of communication and runtime between our ePSU and (PSU with DEL [13], ePSU [16], corrected ePSU [16]) in balanced setting. Both x and y-axis are in log scale. The first figure shows the communication cost increases as the set size increases. The second figure shows the runtime decreases as the bandwidth increases. The last two figures show the runtime increases as the set size increases.

Then, the comparison with scheme [13] with DEL is as follows. Figure 12 and Table 5 show that although our balanced ePSU protocol incurs slightly higher communication compared to [13] with DEL, it achieves shorter runtime across different network environments. Specifically, under LAN, our protocol achieves a clear advantage: for set sizes from 2^{10} to 2^{20}, it speeds up by $1.5 - 11.6\times$. Therefore, our balanced ePSU protocol addresses the DEL issue identified in [13], with only a minor communication sacrifice.

5.2 Performance Evaluation in Unbalanced Setting

We implement our unbalanced ePSU and compare it against state-of-the-art unbalanced PSU with DEL [18] and unbalanced ePSU [16]. In addition, we also implement the corrected unbalanced ePSU from [16]. The results of the communication and runtime comparison in the single thread are presented in Table 6 and Figure 13. First, the comparison with [16] is as follows.

- **Communication.** As shown in Figure 13, our unbalanced ePSU consistently outperforms both the original and corrected versions of [16] in terms of communication. As shown in Table 6, when the sender's set size is fixed at 2^{10} and the receiver's set size grows from 2^{12} to 2^{20}, our protocol maintains a constant communication cost of 1.746 MB, independent of the receiver's set size. In contrast, the communication costs in [18] and [16] increase with the receiver's set size. Notably, compared with the corrected version, as the receiver's set size grows from 2^{12} to 2^{20}, the communication reduction achieved by our protocol increases from about $1.5\times$ up to $45.8\times$. Hence, our advantage becomes increasingly significant with larger the receiver set sizes.
- **Runtime.** As shown in Figure 13, our protocol consistently outperforms both the original and the corrected unbalanced ePSU of [16] in terms of runtime. Table 6 demonstrates that, under various network conditions, as the receiver's set size grows from 2^{12} to 2^{20}, the runtime speedup of our protocol over the unbalanced ePSU of [16] increases from 1.3–1.4× to 1.6–1.8×, while the speedup over the corrected unbalanced ePSU of [16] increases from 1.3–1.6× to 1.6–6.7×.

The comparison with scheme [18] with DEL is as follows.

- **Communication.** Figure 13 and Table 6 show that our communication is only 0.07 MB lower than [18] at $(2^{10}, 2^{12})$, but as the receiver's set size increases we achieve at least $1.5\times$ reduction versus [18]. Thus, our advantage becomes pronounced as the receiver's set size grows.
- **Runtime.** Figure 13 and Table 6 show that that when the sender's set size is fixed at 2^{10}, ours' speedup becomes more pronounced as the receiver's set size increases; specifically, with the receiver's set size at 2^{20}, we achieve $1.6 - 1.8\times$ speedup, depending on the network environment.

Thus, our unbalanced protocol not only realizes better performance, but also addresses the DEL issue identified in [18].

Size($\|X\|$, $\|Y\|$)	Protocols	Comm.(MB)	Total running time (s) with single thread		
			10 Gbps	100 Mbps	10 Mbps
$(2^{10}, 2^{12})$	[18]* (DEL)	**1.614**	**1.538**	**4.496**	**4.511**
	[16]* (DEL-free)	2.237	2.271	10.364	12.152
	Corrected [16] (DEL-free)	2.583	2.405	12.185	14.679
	Our $\Pi^{\text{Unb}}_{\text{ePSU}}$ (DEL-free)	1.746	1.812	7.394	9.214
$(2^{10}, 2^{14})$	[18]* (DEL)	2.681	2.611	**7.449**	**7.463**
	[16]* (DEL-free)	2.237	3.321	14.052	17.031
	Corrected [16] (DEL-free)	3.469	3.802	16.425	21.844
	Our $\Pi^{\text{Unb}}_{\text{ePSU}}$ (DEL-free)	**1.746**	**2.077**	7.463	9.315
$(2^{10}, 2^{16})$	[18]* (DEL)	2.681	3.382	7.981	**8.495**
	[16]* (DEL-free)	2.237	4.549	16.249	19.279
	Corrected [16] (DEL-free)	7.049	6.349	20.551	29.068
	Our $\Pi^{\text{Unb}}_{\text{ePSU}}$ (DEL-free)	**1.746**	**2.373**	**7.696**	9.512
$(2^{10}, 2^{18})$	[18]* (DEL)	2.367	7.638	11.846	12.255
	[16]* (DEL-free)	2.237	7.479	17.471	20.974
	Corrected [16] (DEL-free)	21.510	15.388	30.562	54.369
	Our $\Pi^{\text{Unb}}_{\text{ePSU}}$ (DEL-free)	**1.746**	**5.193**	**9.153**	**11.754**
$(2^{10}, 2^{20})$	[18]* (DEL)	2.767	32.093	36.479	42.425
	[16]* (DEL-free)	2.322	31.627	41.506	46.675
	Corrected [16] (DEL-free)	80.002	67.856	91.924	177.152
	Our $\Pi^{\text{Unb}}_{\text{ePSU}}$ (DEL-free)	**1.746**	**19.423**	**23.498**	**26.289**

Table 6: Comparisons of communication and runtime between our ePSU and (PSU with DEL [18], ePSU [16], corrected ePSU [16]) in unbalanced setting in single thread. 10Gbps bandwidth, 0.2ms RTT; 100Mbps and 10Mbps bandwidth, 80ms RTT; The best and the second results are marked in pink and blue, respectively. "*" denotes the privacy leakage caused by directly applying hash-to-bin on sets.

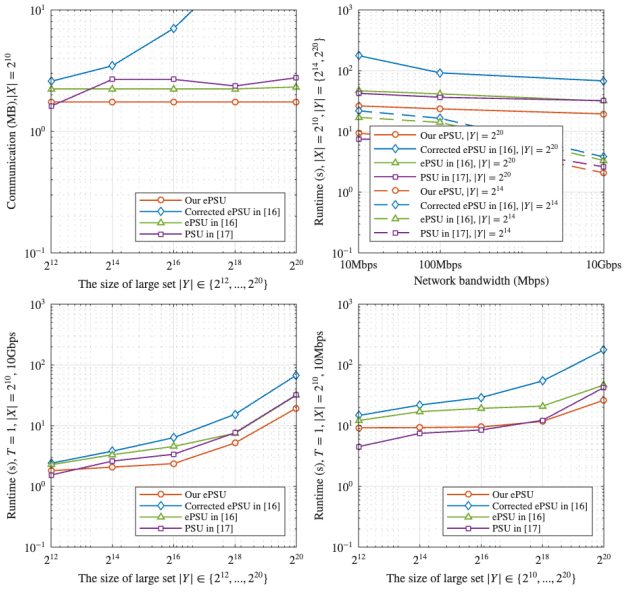

Figure 13: Comparisons of communication and runtime between our ePSU and (PSU with DEL [18], ePSU [16], corrected ePSU [16]) in unbalanced setting. Both x and y-axis are in log scale. The first figure shows the communication cost increases as the set size increases. The second figure shows the runtime decreases as the bandwidth increases. The last two figures show the runtime increases as the set size increases.

References

[1] Arjen Lenstra and Tim Voss. Information Security Risk Assessment, Aggregation, and Mitigation. In *ACISP'04*, 2004.

[2] Kyle Hogan, Noah Luther, Nabil Schear, Emily Shen, David Stott, Sophia Yakoubov, and Arkady Yerukhimovich. Secure Multiparty Computation for Cooperative Cyber Risk Assessment. In *SecDev'16*, 2016.

[3] Heewon Chung, Myungsun Kim, and Patrick Chulsoon Yang. Multiparty Delegated Private Set Union With Efficient Updates on Outsourced Datasets. In *TDSC*, 2025.

[4] Martin Burkhart, Mario Strasser, Dilip Many, and Xenofontas Dimitropoulos. SEPIA: Privacy-Preserving Aggregation of Multi-Domain Network Events and

Statistics. In *USENIX Security'10*, 2010.
[5] Julien Freudiger, Emiliano De Cristofaro, and Alejandro E. Brito. Controlled Data Sharing for Collaborative Predictive Blacklisting. In *DIMVA'15*, 2015.
[6] Sivaramakrishnan Ramanathan, Jelena Mirkovic, and Minlan Yu. BLAG: improving the accuracy of blacklists. In *NDSS'20*, 2020.
[7] Prasad Buddhavarapu, Andrew Knox, Payman Mohassel, Shubho Sengupta, Erik Taubeneck, and Vlad Vlaskin. Private Matching for Compute. In *Cryptology ePrint Archive*, 2020.
[8] Gayathri Garimella, Payman Mohassel, Mike Rosulek, Saeed Sadeghian, and Jaspal Singh. Private Set Operations from Oblivious Switching. In *PKC'21*, 2021.
[9] Jianping Yan, Lifei Wei, Xiansong Qian, Lei Zhang. IDPriU: A two-party ID-private data union protocol for privacy-preserving machine learning. In *J. Inf. Secur. Appl*, 2025.
[10] Vladimir Kolesnikov, Mike Rosulek, Ni Trieu, and Xiao Wang. Scalable Private Set Union from Symmetric-Key Techniques. In *ASIACRYPT'19*, 2019.
[11] Michael O. Rabin. How to exchange secrets with oblivious transfer. In *IACR Cryptol. ePrint Arch.*, 2005.
[12] Yanxue Jia, Shi-Feng Sun, Hong-Sheng Zhou, Jiajun Du, and Dawu Gu. Shuffle-based Private Set Union: Faster and More Secure. In *USENIX Security'22*, 2022.
[13] Cong Zhang, Yu Chen, Weiran Liu, Min Zhang, and Dongdai Lin. Linear Private Set Union from Multi-Query Reverse Private Membership Test. In *USENIX Security'23*, 2023.
[14] Yanxue Jia, Shifeng Sun, Hongsheng Zhou, and Dawu Gu. Scalable Private Set Union, with Stronger Security. In *USENIX Security'24*, 2024.
[15] Xiaojie Guo, Ye Han, Zheli Liu, Ding Wang, Yan Jia, and Jin Li. Birds of a Feather Flock Together: How Set Bias Helps to Deanonymize You via Revealed Intersection Sizes. *USENIX Security'22*, 2022.
[16] Binbin Tu, Yujie Bai, Cong Zhang, Yang Cao, and Yu Chen. Fast Enhanced Private Set Union in the Balanced and Unbalanced Scenarios. In *USENIX Security'25*, 2025.
[17] Gowri R Chandran, Thomas Schneider, Maximilian Stillger, and Christian Weinert. Concretely Efficient Private Set Union via Circuit-Based PSI. In *ASIACCS'25*, 2025.
[18] Binbin Tu, Yu Chen, Qi Liu, and Cong Zhang. Fast Unbalanced Private Set Union from Fully Homomorphic Encryption. In *ACM CCS'23*, 2023.
[19] Hao Chen, Kim Laine, and Peter Rindal. Fast Private Set Intersection from Homomorphic Encryption. In *ACM CCS'17*, 2017.
[20] Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled PSI from Fully Homomorphic Encryption with Malicious Security. In *ACM CCS'18*, 2018.
[21] Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled PSI from homomorphic encryption with reduced computation and communication. In *ACM CCS'21*, 2021.
[22] Cong Zhang, Yu Chen, Weiran Liu, Liqiang Peng, Meng Hao, Anyu Wang, and Xiaoyun Wang. Unbalanced Private Set Union with Reduced Computation and Communication. In *ACM CCS'24*, 2024.
[23] Payman Mohassel and Seyed Saeed Sadeghian. How to hide circuits in MPC an efficient framework for private function evaluation. In *EUROCRYPT'13*, 2013.
[24] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. PIR with Compressed Queries and Amortized Query Processing. In *S&P'18*, 2018.
[25] Muhammad Haris Mughees and Ling Ren. Vectorized Batch Private Information Retrieval. In *S&P'23*, 2023.
[26] Jian Liu, Jingyu Li, Di Wu, and Kui Ren. PIRANA: Faster Multi-query PIR via Constant-weight Codes. In *S&P'24*, 2024.
[27] Melissa Chase and Peihan Miao. Private Set Intersection in the Internet Setting from Lightweight Oblivious PRF. In *CRYPTO'20*, 2020.
[28] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious Key-Value Stores and Amplification for Private Set Intersection. In *CRYPTO'21*, 2021.
[29] Srinivasan Raghuraman and Peter Rindal. Blazing Fast PSI from Improved OKVS and Subfield VOLE. In *ACM CCS'22*, 2022.
[30] Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. NearOptimal Oblivious Key-Value Stores for Efficient PSI, PSU and Volume-Hiding Multi-Maps. In *USENIX Security'23*, 2023.
[31] Ronald Cramer and Victor Shoup. Universal Hash Proofs and a Paradigm for Adaptive Chosen Ciphertext Secure Public-Key Encryption. In *Eurocrypt'02*, 2002.
[32] Hesse Julia, Dennis Hofheinz, and Lisa Kohl. On tightly secure non-interactive key exchange. In *CRYPTO'18*, 2018.
[33] Shohei Egashira, Yuyu Wang, Keisuke Tanaka. Fine-Grained Cryptography Revisited. In *J. Cryptol*, 2021.
[34] Quanyu Zhao, Bingbing Jiang, Yuan Zhang, Heng Wang, Yunlong Mao, and Sheng Zhong. Unbalanced private set intersection with linear communication complexity. In *Sci. China Inf. Sci*, 2024.
[35] Benny Pinkas, Thomas Schneider, and Michael Zohner. Scalable Private Set Intersection Based on OT Extension. *ACM Trans. Priv. Secur*, 2018.
[36] Benny Pinkas, Thomas Schneider, Gil Segev, Michael Zohner. Phasing: Private Set Intersection Using Permutation-based Hashing. In *USENIX Security'25*, 2015.
[37] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, Ni Trieu. Efficient Batched Oblivious PRF with Applications to Private Set Intersection. In *CCS'16*, 2016.
[38] M. Ajtal. Generating hard instances of lattice problems. *STOC'96*, 1996.
[39] https://github.com/encryptogroup/circuitPSU.git
[40] https://github.com/google/private-membership.git
[41] https://zenodo.org/records/14725816
[42] https://github.com/weidai11/cryptopp.git
[43] https://github.com/libntl/ntl.git
[44] https://github.com/openssl/openssl.git
[45] https://github.com/ladnir/cryptoTools
[46] https://github.com/Visa-Research/coproto.git

A Security Proof of $\Pi^{\mathbf{Bal}}_{\mathbf{ePSU}}$

Proof. We will show that for any adversary $\mathcal{A}$, we can construct a simulator *Sim* that can simulate the view of the corrupted the sender and the corrupted the receiver, such that any PPT environment ϵ cannot distinguish the execution in the ideal world from that in the real world.

Corrupted the sender: Sim simulates a real execution in which the sender is corrupted. Since $\mathcal{A}$ is semi-honest, *Sim* can obtain the input of X of the sender directly, and externally send X to $\mathcal{F}^{\mathrm{n,n}}_{\mathrm{ePSU}}$, and then receives $\langle \mathsf{Request}, \mathcal{S} \rangle$. *Sim* randomly selects *key* and simulates the execution of Π_{OPRF}. Once receiving OKVS D from $\mathcal{A}$, *Sim* randomly samples $\{w^*_i\}_{i\in[n]}$ and $\{\alpha_i\}_{i\in[\beta]}$. For each $i \in [n]$ and $j \in [\gamma]$, it computes $s^j_i = \mathsf{Decode}(\mathsf{D}, w^*_i||j)$. It then constructs OKVS $D^* = \mathsf{Encode}((w^*_i||j, \alpha_{h_j(w^*_i)} \oplus s^j_i)_{i\in[n],j\in[\gamma]})$. When receiving $\{m_1, \ldots, m_\beta\}$ and π from $\mathcal{A}$, *Sim* checks if it is a permutation of β items. If so, *Sim* randomly selects $\{c_1, \ldots, c_\beta\}$ and simulates the execution of Π_{pCERG}. After receiving $\{c_1, \ldots, c_\beta\}$ from $\mathcal{A}$, *Sim* randomly selects $\{u_1, \ldots, u_\beta\}$ and simulates the execution of Π_{nECRG}. After obtaining $\{c_i\}_{i\in[\beta]}$, *Sim* sends $\langle \mathsf{Request}, \mathsf{OK} \rangle$ to $\mathcal{F}^{n,n}_{\mathrm{ePSU}}$.

We argue that the outputs of *Sim* are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

Hybrid_0: The view of the sender in the real protocol.

Hybrid_1: Same as Hybrid_0 except that the output of Π_{OPRF} is replaced by *key* chosen by *Sim*, and runs the $\mathcal{F}_{\mathrm{OPRF}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{OPRF} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_0.

Hybrid_2: Same as Hybrid_1 except that computing a random OKVS D^* by *Sim*. Briefly, this simulation is indistinguishable for the following reasons: The randomness of $\{\alpha_{h_j(w^*_i)} \oplus s^j_i\}_{i\in[n],j\in[\gamma]}$ is indistinguishable from random, and the obliviousness and randomness of OKVS guarantees the D^* is distributed uniformly.

Hybrid_3: Same as Hybrid_2 except that the output of Π_{pCERG} is replaced by $\{c_1, \ldots, c_\beta\}$ chosen by *Sim*, and runs the $\mathcal{F}_{\mathrm{pCERG}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{pCERG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_2. The bybrid is the view output by *Sim*.

Hybrid_4: Same as Hybrid_3 except that the output of Π_{nECRG} is replaced by $\{u_1, \ldots, u_\beta\}$ chosen by *Sim*, and runs the $\mathcal{F}_{\mathrm{nECRG}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{nECRG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_3.

Corrupted the receiver: Sim simulates a real execution in which the receiver is corrupted. Since $\mathcal{A}$ is semi-honest, *Sim* can obtain the input of Y of the receiver directly, and externally send Y to $\mathcal{F}^{n,n}_{\mathrm{ePSU}}$ and then receives $\langle \mathsf{Request}, \mathcal{R} \rangle$. Upon receiving Y from $\mathcal{A}$, *Sim* randomly selects $\{w^*_i\}_{i\in[n]}$ and simulates the execution of Π_{OPRF}. *Sim* randomly constructs the cuckoo hash tables $\{X_C, \tilde{X}_C\}$

and $\{r_1, \ldots, r_\beta\}$. It then constructs OKVS D according to the $\tilde{X}_C$ and $\{r_1, \ldots, r_\beta\}$. When receiving $\{\alpha_1, \ldots, \alpha_\beta\}$ from $\mathcal{A}$, *Sim* randomly selects $\{c_1^*, \ldots, c_\beta^*\}$ and simulates the execution of Π_{pECRG}. After receiving $\{c_1^*, \ldots, c_\beta^*\}$ from $\mathcal{A}$, *Sim* randomly selects $\{v_1, \ldots, v_\beta\}$ and simulates the execution of Π_{nCERG}. *Sim* sends $\langle \text{Request}, \text{OK} \rangle$ to $\mathcal{F}_{\text{ePSU}}^{n,n}$. After receiving $\mathsf{Z} = \mathsf{X} \cup \mathsf{Y}$ from $\mathcal{F}_{\text{ePSU}}^{n,n}$, *Sim* computes $\tilde{\mathsf{X}} = \mathsf{X} \backslash \mathsf{Y} = \mathsf{Z} \backslash \mathsf{Y}$ and randomly selects a subset $\tilde{V}$ of $\{v_1, \ldots, v_\beta\}$ where $|\tilde{V}| = |\tilde{\mathsf{X}}|$. For each item $\tilde{v}_i \in \tilde{V}$, *Sim* sets $c_i = \tilde{v}_i \oplus (\tilde{v}_i || \mathsf{H}(\tilde{x}_i))$. Finally, *Sim* randomly chooses $\tilde{v}_i$ for $i \in \{|\tilde{\mathsf{X}}| + 1, \ldots, \beta\}$ and sends all shuffled ciphertexts to $\mathcal{A}$.

We argue that the outputs of *Sim* are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

Hybrid_0: The view of the sender in the real protocol.

Hybrid_1: Same as Hybrid_0 except that the output of Π_{OPRF} is replaced by $\{w_1^*, \ldots, w_n^*\}$ chosen by *Sim*, and runs the $\mathcal{F}_{\text{OPRF}}$ simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{OPRF} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_0.

Hybrid_2: Same as Hybrid_1 except that computing a random OKVS D by *Sim*. Briefly, this simulation is indistinguishable for the following reasons: The randomness of $\{r_1, \ldots, r_\beta\}$ is indistinguishable from random, and the obliviousness and randomness of OKVS guarantees the D is distributed uniformly.

Hybrid_3: Same as Hybrid_2 except that the output of Π_{pECRG} is replaced by $\{c_1, \ldots, c_\beta\}$ chosen by *Sim*, and runs the $\mathcal{F}_{\text{pECRG}}$ simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{pECRG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_2.

Hybrid_4: Same as Hybrid_3 except that the output of Π_{nCERG} is replaced by $\{v_1, \ldots, v_\beta\}$ chosen by *Sim*, and runs the $\mathcal{F}_{\text{nCERG}}$ simulator to produce the simulated view for $\mathcal{R}$. The corresponding $\{c_i\}_{i \in [\beta]}$ are also changed according to $\{v_1, \ldots, v_\beta\}$. The security of protocol Π_{nCERG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_3. The hybrid is the view output by *Sim*. □

B Security Proof of Π_{OECRG}

Proof. After obtaining $\mathbf{c}_i^{\mathcal{S}}$ from the sender, the receiver obtains $\tilde{\mathbf{c}}_i^{\mathcal{S}}$ by computing

$$\begin{aligned}\tilde{\mathbf{c}}_i^{\mathcal{S}} &= \mathbf{sk} \cdot \mathbf{c}_i^{\mathcal{S}} \bmod N = \mathbf{sk}\left(\sum_{k=1}^{\ell} x_{i,k}\mathbf{a}_k + \mathbf{A}\mathbf{w}_i\right) \bmod N \\ &= \mathbf{sk}\sum_{k=1}^{\ell} x_{i,k}\mathbf{a}_k + \mathbf{pk} \cdot \mathbf{w}_i \bmod N \\ &= \mathbf{sk}\sum_{k=1}^{\ell} x_{i,k}\mathbf{a}_k + p_i \bmod N,\end{aligned}$$

where $i \in [m]$. Then, the receiver obtains $z_{i,j}$ by computing

$$\begin{aligned}z_{i,j} &= \tilde{\mathbf{c}}_i^{\mathcal{S}} - \mathbf{c}_i^{\mathcal{R}} = \mathbf{sk}\sum_{k=1}^{\ell} x_{i,k}\mathbf{a}_k + p_i - \text{sk}\sum_{k=1}^{\ell} y_{j,k}\mathbf{a}_k \bmod N \\ &= \mathbf{sk}\sum_{k=1}^{\ell}(x_{i,k} - y_{j,k})\mathbf{a}_k + p_i \bmod N,\end{aligned}$$

where $i \in [m]$ and $j \in [n]$. If $x_i \in X \cap Y$, implying that $x_i = y_j$, therefore $z_{i,j} = p_i$. Then the receiver obtains M_i by computing

$$\begin{aligned}M_i &= g_i^{r_i} \cdot (g_i)^{\prod_{j=1}^{n} z_{i,j}} \bmod N_i \\ &= g_i^{r_i} \cdot (g_i)^{z_{i,1}\ldots z_{i,j-1}p_i z_{i,j+1}\ldots z_{i,n}} \bmod N_i.\end{aligned}$$

The receiver sends $\{M_1, \ldots, M_m\}$ to the sender, and the sender computes $t_i = \Phi(N_i)/p_i$, $R_i = g^{t_i} \bmod N_i$ and M_i^*, where

$$\begin{aligned}M_i^* = M_i^{t_i} = (g_i^{r_i \Phi(N_i)/p_i \bmod \Phi(N_i)}) \\ (g_i)^{(\Phi(N_i) \bmod \Phi(N_i))z_{i,1}\ldots z_{i,j-1}z_{i,j+1}\ldots z_{i,n}} \\ = (g_i)^{r_i \Phi(N_i)/p_i \bmod \Phi(N_i)}.\end{aligned}$$

If $x_i \neq y_j$, then $z_{i,j} \neq p_i$, therefore

$$\begin{aligned}M_i^* = M_i^{t_i} = g_i^{r_i t_i}(g_i)^{t_i \sum_{j=1}^{n} z_{i,j}} \bmod N_i \\ = (g_i)^{r_i \Phi(N_i)/p_i \bmod \Phi(N_i)} \\ (g_i)^{(\Phi(N_i)/p_i \bmod \Phi(N_i))z_{i,1}\ldots z_{i,j-1}z_{i,j}z_{i,j+1}\ldots z_{i,n}}.\end{aligned}$$

Define events A as "r_i/p_i is an integer" and B as "r_i/p_i is not an integer". Since there are N_i items in $\mathbb{Z}_{N_i}$ and about N_i/p_i of them are divisible by p_i, we have

$$\begin{aligned}Pr[A] &= \frac{\frac{N_i}{p_i}}{N_i} = \frac{1}{p_i}, \\ Pr[B] &= 1 - Pr[A] = 1 - \frac{1}{p_i}.\end{aligned}$$

Because p_i is a prime under κ security parameter, $Pr[A] \ll 2^{-\lambda}$, which is negligible and $Pr[B]$ is effectively 1. Similarly, define event C as "For $j \in [n]$, all $z_{i,j}/p_i$ values are not integers" and event D as "For $j \in [n]$, at least one of $z_{i,j}/p_i$ values is an integer". Therefore, we have

$$\begin{aligned}Pr[C] &= (1 - \frac{\frac{N_i}{p_i}}{N_i})^n = (1 - \frac{1}{p_i})^n < 1 - \frac{n}{p_i}, \\ Pr[D] &= 1 - Pr[C] = \frac{n}{p_i}.\end{aligned}$$

Under the usual assumption that $n \ll 2^{\kappa}$, we have $n/p_i \ll 2^{-\lambda}$, which is negligible. Therefore $Pr[C]$ is effectively 1 and $Pr[D]$ is negligible. Therefore, $Pr[(g_i)^{r_i \Phi(N_i)/p_i \bmod \Phi(N_i)} = 1]$ and $Pr[(g_i)^{(\Phi(N_i)/p_i \bmod \Phi(N_i))z_{i,1}\ldots z_{i,j-1}z_{i,j}z_{i,j+1}\ldots z_{i,n}} = 1 | z_{i,j} \neq p_i \text{ for } j \in [n]]$ are both negligible.

Now, we will present that for any adversary $\mathcal{A}$, we can construct a simulator *Sim* that can simulate the view of the corrupted the sender and the corrupted the receiver, such that any PPT environment ϵ cannot distinguish the execution in the ideal world from that in the real world.

Corrupted the sender: Sim simulates a real execution in which the sender is corrupted. Since $\mathcal{A}$ is semi-honest, *Sim* can obtain the input of X directly, and externally sends the set X to $\mathcal{F}_{\text{OECRG}}$, and then receives $\langle \text{Request}, \mathcal{S} \rangle$. *Sim* simulates the receiver's HPS.KeyGen(1^κ) to produce $(\mathbf{sk}, \mathbf{pk}, \mathbf{A}, N)$, and randomly samples $\{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\}$. It then follows the protocol's steps to simulate the receiver's computation of $\{M_1, \ldots, M_m\}$. After obtaining $\{R_i\}_{i \in [m]}$ and $\{M_i^*\}_{i \in [m]}$, *Sim* sends $\langle \text{Request}, \text{OK} \rangle$ to $\mathcal{F}_{\text{OECRG}}$.

We argue that the outputs of *Sim* are indistinguishable from the real view of the sender by the following hybrids:

Hybrid_0: The view of the sender in the real protocol.

Hybrid_1: Same as Hybrid_0 except that *Sim* generates the corresponding parameters of the sender, which is $(\mathbf{sk}, \mathbf{pk}, \mathbf{A}, N)$ and random values $\{\mathbf{a}_1, \ldots, \mathbf{a}_\ell\}$. The security of HPS and the randomness of $\{\mathbf{a}_i\}_{i\in[\ell]}$ guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_0.

Hybrid_2: Same as Hybrid_1 except that *Sim* chooses random values $\{M_i\}_{i\in[m]}$ from $\mathbb{Z}_{N_i}$. This hybrid is computationally indistinguishable from Hybrid_1 by the RSA assumption and HPS. The hybrid is the view output by *Sim*.

Corrupted the receiver: Sim simulates a real execution in which the receiver is corrupted. Since $\mathcal{A}$ is semi-honest, *Sim* can obtain the input of Y directly, and externally sends the set Y to $\mathcal{F}_{\text{OECRG}}$, and then receives $\langle \text{Request}, \mathcal{R}\rangle$. *Sim* simulates the sender's $\text{RSA.KeyGen}(1^\kappa)$ to produce $\{p_i, q_i, N_i, g_i\}_{i\in[m]}$. It then follows the protocol's steps to simulate the sender's computation of $\{\mathbf{c}_1^S, \ldots, \mathbf{c}_m^S\}$, and $\{M_1^*, \ldots, M_m^*\}$. *Sim* sends $\langle \text{Request}, \text{OK}\rangle$ to $\mathcal{F}_{\text{OECRG}}$.

We argue that the outputs of *Sim* are indistinguishable from the real view of the sender by the following hybrids:

Hybrid_0: The view of the sender in the real protocol.

Hybrid_1: Same as Hybrid_0 except that *Sim* generates the corresponding parameters of the receiver, which is $\{p_i, q_i, N_i, g_i\}_{i\in[m]}$. The security of RSA assumption guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_0.

Hybrid_2: Same as Hybrid_1 except that *Sim* chooses random values $\{\mathbf{c}_1^S, \ldots, \mathbf{c}_m^S\}$ from $\mathbb{Z}_N$. This hybrid is computationally indistinguishable from Hybrid_0.

Specifically, the simulated witness vector $\{\mathbf{w}_1, \ldots, \mathbf{w}_m\}_{i\in[m]}$ by *Sim* is computationally indistinguishable from the real. Under the leftover hash lemma and entropy smoothing lemma [38], *Sim*'s simulated $\{\mathbf{c}_1^S, \ldots, \mathbf{c}_m^S\}$ is also computationally indistinguishable from the real.

Hybrid_3 : Same as Hybrid_2 except that *Sim* chooses random values $\{M_i^*\}_{i\in[m]}$ from $\mathbb{Z}_{N_i}$. This hybrid is computationally indistinguishable from Hybrid_2 by the RSA assumption. The hybrid is the view output by *Sim*.

□

C Security Proof of $\Pi_{\text{ePSU}}^{\text{Unb}}$

Proof. We will present that for any adversary $\mathcal{A}$, we can construct a simulator *Sim* that can simulate the view of the corrupted the sender and the corrupted the receiver, such that any PPT environment ϵ cannot distinguish the execution in the ideal world from that in the real world.

Corrupted the sender: Sim simulates a real execution in which the sender is corrupted. Since $\mathcal{A}$ is semi-honest, *Sim* can obtain the input of X directly, and externally sends the set X to $\mathcal{F}_{\text{ePSU}}^{m,n}$, . When receiving X from $\mathcal{A}$, *Sim* randomly selects $\{R_1, \ldots, R_m\}$, and simulates the execution of Π_{OECRG}. After receiving $\{R_1, \ldots, R_m\}$ from $\mathcal{A}$, *Sim* randomly selects $\{t_1, \ldots, t_m\}$, and simulates the execution of Π_{nECRG}. After obtaining $\{v_i, v_i^*\}_{i\in[m]}$, *Sim* sends $\langle \text{Request}, \text{OK}\rangle$ to $\mathcal{F}_{\text{ePSU}}^{m,n}$.

We argue that the outputs of *Sim* are indistinguishable from the real view of the sender by the following hybrids:

Hybrid_0: The view of the sender in the real protocol.

Hybrid_1: Same as Hybrid_0 except that the output of Π_{OECRG} is replaced by $\{R_1, \ldots, R_m\}$ chosen by *Sim*, and *Sim* runs the $\mathcal{F}_{\text{OECRG}}$ simulator to produce the simulated view of the sender. The security of protocol Π_{OECRG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_0.

Hybrid_2: Same as Hybrid_1 except that the output of Π_{nECRG} is replaced by $\{t_1, \ldots, t_m\}$ chosen by *Sim*, and *Sim* runs the $\mathcal{F}_{\text{nECRG}}$ simulator to produce the simulated view of the sender. The security of protocol Π_{nECRG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_1. The hybrid is the view output by *Sim*.

Corrupted the receiver: Sim simulates a real execution in which the receiver is corrupted. Since $\mathcal{A}$ is semi-honest, *Sim* can obtain the input of Y directly, can externally send the set Y to $\mathcal{F}_{\text{ePSU}}^{m,n}$, and then receives $\langle \text{Request}, \mathcal{R}\rangle$. When receiving Y from $\mathcal{A}$, *Sim* randomly selects $\{R_1^*, \ldots, R_m^*\}$, and simulates the execution of Π_{OECRG}. After receiving $\{R_1^*, \ldots, R_m^*\}$ from $\mathcal{A}$, *Sim* randomly selects $\{t_1^*, \ldots, t_m^*\}$, and simulates the execution of Π_{nECRG}. *Sim* sends $\langle \text{Request}, \text{OK}\rangle$ to $\mathcal{F}_{\text{ePSU}}^{m,n}$. After receiving $\mathsf{Z} = \mathsf{X} \cup \mathsf{Y}$ from $\mathcal{F}_{\text{ePSU}}^{m,n}$, *Sim* computes $\mathsf{X}^* = \mathsf{X}\backslash\mathsf{Y} = \mathsf{Z}\backslash\mathsf{Y}$ and randomly selects a subset $\tilde{T}$ of $\{t_1^*, \ldots, t_\beta^*\}$ where $|\tilde{T}| = |\mathsf{X}^*|$. For each item $\tilde{t}_i \in \tilde{T}$, *Sim* sets $v_i = \tilde{t}_i \oplus (x_i^* || \mathsf{H}(x_i^*))$. Finally, *Sim* randomly chooses t_i for $i \in \{|X^*| + 1, \ldots, \beta\}$ and sends all shuffled ciphertexts to $\mathcal{A}$.

We argue that the output of *Sim* are indistinguishable from the real view of the receiver by the following hybrids:

Hybrid_0: The view of the receiver in the real protocol.

Hybrid_1: Same as Hybrid_0 except that the output of Π_{OECRG} is replaced by $\{R_1^*, \ldots, R_m^*\}$ chosen by *Sim*, and *Sim* runs the $\mathcal{F}_{\text{OECRG}}$ simulator to produce the simulated view of the receiver. The security of protocol Π_{OECRG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_0.

Hybrid_2: Same as Hybrid_1 except that the output of Π_{nECRG} is replaced by $\{t_1^*, \ldots, t_m^*\}$ chosen by *Sim*, and *Sim* runs the $\mathcal{F}_{\text{nECRG}}$ simulator to produce the simulated view of the receiver. The corresponding $\{v_i\}_{i\in[m]}$ are also changed according to $\{t_1^*, \ldots, t_m^*\}$. The security of protocol Π_{nECRG} guarantees the view in simulation is computationally indistinguishable from the view in Hybrid_1. The bybrid is the view output by *Sim*.

□

D Related Work

Balanced PSU. In 2019, Kolesnikov et al. [10] proposed the first scalable PSU protocol. They introduced RPMT and combined with OT to compute the set union. RPMT secretly determines, for each item x_i ($i \in [n]$) from the sender's set X, whether $x_i \in$ Y: if $x_i \in$ Y, the receiver obtains only a dummy value during the OT phase; otherwise, the receiver receives the item x_i. After n invocations of RPMT and OT, the receiver obtains the union $\mathsf{X} \cup \mathsf{Y}$. Their protocol employs a bucketing technique to group items, achieving $O(n \log n)$ communication complexity and $O(n \log n \log\log n)$ computation complexity.

In 2021, Garimella et al. [8] introduced a PSU protocol based on permuted characteristic (PC), achieving $O(n \log n)$ communication and computation complexities. Their PC protocol is built on Permuted+Share (PS) [23], and provides the batching RPMT functionality. Following, Jia et al. [12] proposed their shuffled-based protocol with the performance comparable to that in [8], achieving identical complexity. Zhang et al. [13] introduced the first balanced

PSU protocol with linear complexity in 2023. They introduced a new protocol for realizing the functionality of batching RPMT, called multi-query RPMT (mq-RPMT).

Jia et al. [14] addressed the DEL problem in works [8, 10, 12, 13] by proposing an ePSU protocol with $O(n \log n)$ computation and communication complexities in 2024. Their construction relies on an Oblivious programmable PRF (OPPRF), PS, and a newly introduced primitive called batched equality-conditional randomness generation (bECRG). While their protocol's runtime is comparable to [13] in LAN setting, its communication overhead is $5-6\times$ higher. Recently, Tu et al. [16] introduced the first balanced ePSU protocol with linear complexity, building on their pnMCRG, and achieving significant performance improvements over [14].

In addition, Chandran et al. [17] observed that directly applying hash-to-bin suffer from privacy leakage: once the receiver obtains $X \setminus Y$, it can further infer which items in Y are not in the intersection. To mitigate this issue, they proposed the use of OPRF together with a Combine-and-Permute featuring a shuffle. However, their core idea relies on realizing mq-RPMT via a circuit-based PSI, which inherently incurs DEL, while performance remains comparable to that of [13]. Notably, the privacy leakage issue stemming from the hash-to-bin, as identified in their work, also exists in the series of protocols proposed in [8, 10, 12, 16].

Unbalanced PSU. In 2023, Tu et al. [18] proposed the first unbalanced PSU protocol with DEL, achieving sublinear communication complexity in the size of the larger set. Their main idea is to adapt FHE based unbalanced private set intersection protocol [19] to the PSU setting. However, the use of bin partitioning techniques still incurs a large constant factor in the protocol's asymptotic complexity. Zhang et al. [22], in 2024, proposed an improved unbalanced PSU with DEL aimed at enhancing performance. Based on a sparse OKVS and batch private information retrieval (Batch PIR) [24–26], it achieves sublinear communication complexity in the larger set size. Although their protocol achieves $20\times$ reduction in communication and $3-10\times$ speedup in runtime during the online phase compared to [18], its expensive offline phase costs require more than ten executions to amortize. In 2025, Tu et al. [16] proposed the first unbalanced ePSU protocol, built on their pnMCRG. However, its communication still remains sublinear in the larger set size and fails to achieve linear based solely on the smaller set size. Similarly, since protocols in [16, 18, 22] directly apply hash-to-bin to sets, inevitably suffer from potential privacy leakage introduced by hash-to-bin.